A Project Stage-1
Report On

DYNAMIC CONTENT RECOMMENDATION CONSIDERING CACHE STATE AND PREFERENCE USING REINFORCEMENT LEARNING

Submitted to

Kommuri Pratap Reddy Institute of Technology (Autonomous)

(Affiliated to JNTUH, Approved by AICTE & Accredited by NAAC & NBA)

In partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

In

Computer Science and Engineering

By

P. Sahithya (23RA1A05B9)

G. Pooja Nandini (23RA1A0596)

M. Sandeep (23RA1A05D0)

B. Rishi Roy (23RA1A05B5)

Under the guidance of

Mrs. C. Vinitha Reddy

M.Tech

Assistant Professor

KPRIT

Imagine. Innovate. Inspire.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

KOMMURI PRATAP REDDY INSTITUTE OF TECHNOLOGY

(UGC AUTONOMOUS)

(Affiliated to JNTUH, Ghanpur (V), Ghatkesar (M), Medchal (D)-500088)

(2023-2027)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

KOMMURI PRATAP REDDY INSTITUTE OF TECHNOLOGY

(UGC AUTONOMOUS)

(Ghanpur (V), Ghatkesar (M), Medchal (D)-500088)

(AFFILIATED TO JNTU, Hyderabad)

CERTIFICATE

This is to certify that the Project Stage-1 entitled **"DYNAMIC CONTENT RECOMMENDATION CONSIDERING CACHE STATE AND PREFERENCE USING REINFORCEMENT LEARNING"** being submitted by **P. Sahithya (23RA1A0502), G. Pooja Nandini (23RA1A0596), M. Sandeep (23RA1A05D0), B. Rishi Roy (23RA1A05B5)** in partial fulfillment for the award of Bachelor of Technology in Computer Science and Engineering to the Kommuri Pratap Reddy Institute of Technology (Autonomous)(Affiliated to JNTUH, Approved by AICTE & Accredited by NAAC & NBA) is a record of confined work carried out by them under my guidance and supervision..

Guide's signature
Mrs. C. VINITHA REDDY
M.Tech
Asst. Professor
Dept. of CSE

Signature of the Head of the Dept.
Dr. K.VAMSHEE KRISHNA
M.Tech, Ph.D.
Professor
HOD of CSE

Place: Ghanpur

Date:

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

KOMMURI PRATAP REDDY INSTITUTE OF TECHNOLOGY

(UGC AUTONOMOUS)

(Ghanpur (V), Ghatkesar (M), Medchal (D)-500088)

(AFFILIATED TO JNTU, Hyderabad)



DECLARATION

We, **P. SAHITHYA (23RA1A05B9), G. POOJA NANDINI (23RA1A0596), M. SANDEEP (23RA1A05D0), B. RISHI ROY (23RA1A05B5)** hereby declare that the project stage-1 report titled **"DYNAMIC CONTENT RECOMMENDATION CONSIDERING CACHE STATE AND PREFERENCE USING REINFORCEMENT LEARNING"** under the guidance of Mrs. C. VINITHA REDDY, Assistant Professor, Department of Computer Science and Engineering, Kommuri Pratap Reddy Institute of Technology, Ghanpur, is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

This is a record of bonafide work carried out by us and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this thesis have not been submitted to any other University and Institute for the award of any Degree or Diploma.

P. SAHITHYA	**(23RA1A05B9)**
G. POOJA NANDINI	**(23RA1A0596)**
M. SANDEEP	**(23RA1A05D0)**
B. RISHI ROY	**(23RA1A05B5)**

ACKNOWLEDGEMENT

We manifest our heartier thankfulness pertaining to our contentment over **Mrs. C. VINITHA REDDY**, Assistant Professor as a project stage-1 guide with whose adroit concomitance the excellence has been exemplified in bringing out this project work with artistry. We express our gratitude to **Dr. K. Vamshee Krishna**, Head of the Department for the encouragement and assistance to us, which contribute to the successful completion of this project. This Acknowledgement will be incomplete without mentioning our sincere gratefulness to our honorable Chairman **Sri. Kommuri Pratap Reddy**, our CEO, **Prof. C.D. Naidu,** and Principal who have been observing posing valiance in abundance, forwarding our individuality to acknowledge our project work tendentiously. At the outset we thank teaching and non-teaching staff of Dept. of CSE for providing us with good faculty and for their moral support throughout the course.

A heartful and sincere gratitude to our beloved parents for their tremendous motivation and moral support. We also express the overall exhilaration and gratitude to all those who animated our project work and accentuated our stance.

P. SAHITHYA	**(23RA1A05B9)**
G. POOJA NANDINI	**(23RA1A0596)**
M. SANDEEP	**(23RA1A05D0)**
B. RISHI ROY	**(23RA1A05B5)**

Vision of the Institute

To emerge as a premier institute for high quality professional graduates who can contribute to economic and social developments of the Nation.

Mission of the Institute

Mission	Statement
IM1	To have holistic approach in curriculum and pedagogy through industry interface to meet the needs of Global Competency.
IM2	To develop students with knowledge, attitude, employability skills, entrepreneurship, research potential and professionally ethical citizens.
IM3	To contribute to advancement of Engineering & Technology that would help to satisfy the societal needs.
IM4	To preserve, promote cultural heritage, humanistic values and spiritual values thus helping in peace and harmony in the society.

Vision of the Department

To Provide Quality Education in Computer Science for the innovative professionals to work for the development of the nation.

Mission of the Department

DM1	Laying the path for contemporary skills in Computer Science through the basic knowledge of mathematics and fundamentals of engineering.
DM2	Provide latest tools and technology to the students as a part of learning Infrastructure.
DM3	Training the students towards employability and entrepreneurship to meet the societal needs.
DM4	Grooming the students with professional and social ethics.

Program Educational Objectives:

PEO's	STATEMENT
PEO1	The Graduate will continuously adapt to evolving technologies in Computer Science and Engineering, staying proficient in contemporary skills through lifelong learning.
PEO2	The graduate will prove an effective communicator as team member and have skills to develop products, offer services and innovation for real-world challenges.
PEO3	The Graduate will develop a conceptual knowledge of industry processes, tools and technologies, equipping them for professional excellence and adaptability in a dynamic technological landscape which contributes to the society at large, without disturbing the ecological balance.

Program Specific Outcomes (PSOs)

PSO1: Competent to apply mathematical methods along with basic sciences and engineering sciences as a multi-disciplinary approach for solving complex engineering problems with due consideration to environment and societal aspects.

PSO2: Provide solutions for real world problems by using modern tool usage with a broad range of programming languages and open source platforms in various computing domains for sustainable development in ethical manner.

PSO3: Implement industry-standard principles, practices and strategies for software development lifecycle and possess proficient skills and knowledge of software design process including system modelling and performance optimization.

ABSTRACT

The rapid growth of content-based services such as video streaming and digital media platforms has created a strong need for intelligent, personalized, and efficient content recommendation systems. Traditional recommendation systems mainly focus on user preferences, such as viewing history, likes, genres, and interests, but they generally do not consider whether the recommended content is already available in the cache. Ignoring the cache state can result in higher content delivery latency, increased bandwidth consumption, and additional network load. To address these challenges, this project proposes a **Dynamic Content Recommendation System Considering Cache State and Preference Using Reinforcement Learning**. The system collects user interaction data to understand individual preferences and monitors the cache state of available content. A Reinforcement Learning agent uses both user preference and cache information to dynamically select suitable content for recommendation. The agent learns from user feedback, such as clicks, viewing behavior, and positive or negative responses, and improves its future decisions through a reward-based learning mechanism. The system also evaluates important performance factors such as recommendation relevance, cache hit ratio, content delivery latency, and bandwidth usage. By combining personalized recommendation with cache-aware decision-making, the proposed system aims to provide more relevant content while improving delivery efficiency and reducing unnecessary network resources. This approach can serve as a prototype for intelligent content delivery systems where personalization and efficient cache utilization are considered together. The proposed system uses Reinforcement Learning to provide personalized content recommendations by considering both user preferences and cache availability. It aims to improve cache utilization, reduce delivery latency, and optimize bandwidth usage. The system dynamically selects relevant content based on user interests and current cache conditions. It learns continuously from user interactions and feedback using a reward-based mechanism. This helps balance personalization and efficient content delivery. The proposed approach aims to improve user experience, cache utilization, and overall system performance.

Key Words: *Reinforcement Learning, Content Recommendation, User Preferences, Cache State, Cache Hit Ratio, Content Delivery, Latency Optimization, Bandwidth Optimization, Personalized Recommendation*.

INDEX

S.NO	CONTENT	PAGE NO.
1.	INTRODUCTION	1
	1.1 INTRODUCTION	1
	1.2 BACKGROUND OF THE PROJECT	2
	1.3 PROBLEM STATEMENT	3
	1.4 OBJECTIVES OF THE PROJECT	3
	1.5 SCOPE OF THE PROJECT	3
2.	LITERATURE SURVEY	4
	2.1 INTRODUCTION	4
	2.2COLLABORATIVE FILTERING -BASED RECOMMENDATION	4
3.	EXISTING SYSTEM	5
	3.1 INTRODUCTION	5
	3.2 DISADVANTAGES	6
4.	PROPOSED SYSTEM	7
	4.1 INTRODUCTION	8
	4.2 WORKING	9
5.	SYSTEM ARCHITECTRE	10-11
6.	MODULES	12-13
7.	UML DIAGRAMS, DFDs	14-19
8.	REFERENCES	20-21
9.	BASE PAPER	22

1.INTRODUCTION

1.1 Introduction

The rapid growth of digital platforms has resulted in a massive increase in the amount of online content available to users. Video streaming platforms, news applications, music services, e-learning systems, and other content delivery platforms provide users with a large variety of content to choose from. As the amount of available content continues to increase, finding suitable and relevant content quickly has become an important challenge. Recommendation systems play an important role in addressing this problem by helping users discover content according to their interests, preferences, and previous interactions. Traditional recommendation systems commonly use techniques such as collaborative filtering, content-based filtering, and hybrid recommendation methods. Collaborative filtering recommends content based on the behavior and preferences of users with similar interests, while content-based filtering recommends items by analyzing the characteristics of content and comparing them with the user's previous interests. Hybrid recommendation systems combine multiple recommendation techniques to improve the quality of recommendations. These approaches have been widely used because they can provide personalized suggestions and reduce the effort required for users to search through large collections of content. Although conventional recommendation systems can provide effective personalization, they mainly focus on factors such as user preferences, historical interactions, ratings, viewing history, and content characteristics. They generally do not consider the current state of the content delivery environment while making recommendation decisions. In particular, the availability of recommended content in the cache is often not considered. As a result, a recommendation system may select content that is highly relevant to the user without considering whether that content can be delivered efficiently from the available cache. Caching is widely used in modern content delivery systems to improve the efficiency of content delivery. Frequently accessed content is temporarily stored in cache locations that are closer to users. When requested content is available in the cache, it can be delivered without repeatedly retrieving it from the origin server. This can help reduce response time, network traffic, and server workload. However, conventional recommendation and caching mechanisms are often designed and operated as separate processes. The recommendation system focuses mainly on user relevance, while the caching mechanism focuses on content storage and delivery efficiency. This separation can create several challenges.

1.2 Background of the Project

Content recommendation systems are widely used in online platforms to suggest content based on user interests. Traditional recommendation systems mainly focus on user preferences and previous activities. However, they may not consider whether the recommended content is already available in the cache. Cache memory stores

frequently accessed content so that it can be delivered quickly without repeatedly fetching it from the main server. By combining user preferences with cache information, the system can provide relevant content with better delivery performance. Reinforcement Learning is a machine learning technique in which an agent learns through actions and rewards. In this project, the Reinforcement Learning agent learns from user interactions and improves the recommendation process over time.

1.3 Problem Statement

Traditional content recommendation systems mainly focus on user preferences and content relevance. They may not consider the current cache availability while selecting recommendations.

When recommended content is not available in the cache, the system needs to fetch it from the server, which may increase response latency and bandwidth usage.

Therefore, there is a need for a recommendation system that considers both user preferences and cache state to provide relevant content with reduced response time.

1.4 Objectives of the Project

The main objectives of this project are:

1. To analyze user preferences based on their interactions and activities.
2. To check the cache availability of requested content.
3. To use Reinforcement Learning for improving content recommendations.
4. To provide personalized recommendations based on user interests.
5. To reduce response latency by considering cache availability.
6. To improve cache hit ratio and system performance.
7. To improve user engagement and satisfaction.

1.5 Scope of the Project

The scope of this project is to develop a dynamic content recommendation system that considers user preferences and cache availability.

The system focuses on the following areas:

- Collecting user interaction data.
- Analyzing user preferences.
- Checking content availability in the cache.
- Applying Reinforcement Learning for recommendation decisions.
- Providing personalized content recommendations.
- Evaluating system performance using response latency, cache hit ratio, and recommendation relevance.

2.LITERATURE SURVEY

2.1 Introduction

Research in recommendation systems has traditionally focused on understanding user interests and predicting content that is likely to be relevant to users. With the rapid growth of digital platforms, recommendation systems have become important for helping users discover suitable content from large collections. Traditional approaches mainly use user preferences, historical interactions, ratings, browsing behavior, and content characteristics. More recent research has explored Reinforcement Learning and system-aware techniques to make recommendation systems more adaptive and efficient.

2.2 Collaborative Filtering-Based Recommendation

Collaborative filtering is one of the most widely used techniques in recommendation systems. It recommends items based on the preferences and interactions of users with similar behavior. User-based collaborative filtering identifies users with similar interests, while item-based collaborative filtering identifies relationships between items based on user interactions. Information such as ratings, clicks, views, and browsing history can be used to generate recommendations. Although collaborative filtering provides personalized recommendations, it mainly focuses on the relationship between users and content. It generally does not consider system-level factors such as cache availability, network latency, content retrieval cost, or server workload. Therefore, a highly relevant recommendation may still require additional retrieval if the content is not available in the cache.

2.3 Content-Based Recommendation

Content-based recommendation methods recommend items by analyzing their characteristics and comparing them with the user's interests. The system uses information such as categories, topics, keywords, genres, descriptions, and previous viewing history. For example, a news platform may recommend articles related to topics previously read by the user, while a video platform may recommend videos from preferred genres. The main advantage of content-based recommendation is its ability to provide personalized results based on individual user interests. However, it may repeatedly recommend similar content and provide limited diversity. It may also take time to adapt when user interests change. In addition, content-based methods generally focus on content relevance and do not consider the current cache state or delivery conditions.

2.4 Hybrid Recommendation Systems

Hybrid recommendation systems combine two or more recommendation techniques to overcome the limitations of individual methods. A common approach combines collaborative filtering with content-based recommendation. This allows the system to use both user interaction patterns and content characteristics to improve recommendation quality. Although hybrid systems can provide better personalization, the recommendation process is usually handled separately from the content delivery.

2.5 Cache-Aware Content Delivery

Caching is widely used in content delivery systems to improve the efficiency of accessing frequently requested content. Frequently accessed content can be temporarily stored closer to users, such as in edge or local cache servers. When the requested content is available in the cache, it can be delivered without repeatedly retrieving it from the origin server. This can help reduce response time, network traffic, and server workload. Traditional cache management techniques commonly consider factors such as content popularity, request frequency, cache capacity, and recent access patterns.

2.6 Reinforcement Learning for Recommendation

Reinforcement Learning is useful for dynamic recommendation because an agent can learn through interaction with users and the recommendation environment. The agent observes the current state, selects an action, receives a reward, and updates its strategy. The state may include user preferences, previous interactions, and content information, while the reward can be based on clicks, ratings, or engagement. However, many existing approaches mainly focus on user engagement .

2.7 System-Aware and Resource-Aware Recommendation

Recent recommendation approaches also consider system conditions along with user preferences. Factors such as network conditions, latency, bandwidth, content availability, and server workload can affect recommendation efficiency. Considering these factors can help balance personalization with resource utilization. However, adaptive approaches that combine user preferences with content delivery conditions are still needed.

2.8 Integration of User Preference and Cache State

User preference indicates what content is relevant to the user, while cache state indicates whether the content is available for efficient delivery. Considering only user preference may result in recommendations that require additional retrieval, while considering only cache availability may result in less relevant content. Therefore, combining both factors can provide a more balanced recommendation decision. Reinforcement Learning can help learn this balance dynamically based on the current system state.

2.9 Research Gap

Existing recommendation systems mainly focus on user preferences and content relevance, while caching techniques focus on content popularity and efficient storage. These processes are often handled separately. Although Reinforcement Learning provides an adaptive recommendation approach, many existing methods do not sufficiently consider cache availability and delivery efficiency. Therefore, there is a research gap in jointly considering user preference and cache state. The proposed project addresses this gap by using Reinforcement Learning to balance

3.EXISTING SYSTEM

3.1 Introduction

Traditional content recommendation systems are designed to identify and suggest content that is relevant to users based mainly on their preferences, historical interactions, and content characteristics. These systems are widely used in video streaming services, news applications, music platforms, e-learning websites, and other content-based applications. The main objective of a recommendation system is to reduce the effort required by users to find suitable content from a large collection of available items. By analyzing user behavior and preferences, the system attempts to provide content that is likely to be useful or interesting to the user. Collaborative Filtering is one of the commonly used recommendation approaches. It recommends content based on the behavior and preferences of users with similar interests. If different users have interacted with similar content, the system can use these patterns to recommend items that may be preferred by a particular user. Content-Based Filtering is another commonly used approach. It generates recommendations by analyzing the characteristics of available content and comparing them with the user's previous interests or interactions. Hybrid Recommendation Systems combine two or more recommendation techniques, such as collaborative filtering and content-based filtering, to improve recommendation quality and overcome some limitations of individual approaches. Although these approaches can provide personalized recommendations, they primarily focus on the relationship between the user and the content. Factors related to the current content delivery environment are generally not included in the recommendation decision. In particular, the cache state of the system is often not considered while selecting content for recommendation. The recommendation system may determine which content is relevant without checking whether that content is already available in the cache. Caching is commonly used in content delivery systems to temporarily store frequently accessed content closer to users. When requested content is available in the cache, it can be delivered without retrieving it again from the origin server. This helps reduce response time, network traffic, and server work load However, in conventional systems, the recommendation process and caching process are usually handled as separate components.

For example, a recommendation system may identify a particular video, article, or learning resource as highly relevant to a user without checking whether the content is currently available in the cache. If the recommended content is not cached, an additional request may be required to retrieve it from the origin server. Such additional retrieval can increase content delivery time and network usage, particularly when the same situation occurs for a large number of users. This can also increase the workload on the origin server. Similarly, a caching mechanism may decide which content should remain available based on factors such as popularity, frequency of requests, or recently accessed content. These decisions may not take individual user preferences into account. As a result, content that is highly relevant to a particular

user may not always be readily available in the cache, while some cached content may have limited relevance to the current user. Another limitation of conventional recommendation systems is their limited ability to respond to changing user interests. User

3.2 Disadvantages of Existing System

- Cache State is Not Considered: Traditional recommendation systems mainly focus on user preferences, historical interactions, and content characteristics. They generally do not consider whether the recommended content is currently available in the cache.
- Increased Content Retrieval: When recommended content is not available in the cache, it may need to be retrieved from the origin server. This can result in additional content requests and unnecessary data transfer.
- Higher Network Latency: Content that needs to be fetched from a distant origin server may take more time to reach the user compared with content already available in a nearby cache.
- Increased Server Workload: Repeated requests for content that is not cached can increase the workload of the origin server, especially when many users request content at the same time.
- Inefficient Cache Utilization: Cached content may not always match the current interests of a particular user. At the same time, relevant content that could

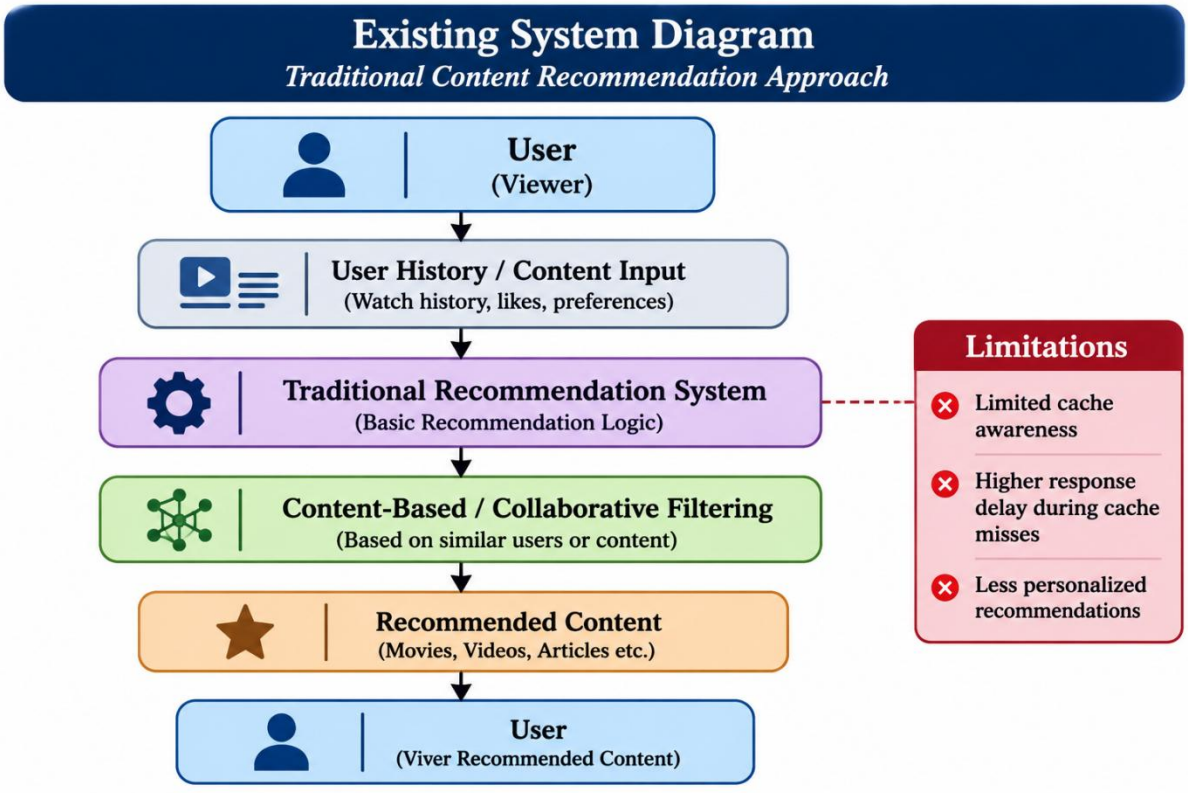


Figure 3: Existing System

4.PROPOSED SYSTEM

4.1 Introduction

The proposed system is designed to overcome the limitations of traditional content recommendation systems by considering both user preferences and cache availability.In this system, Reinforcement Learning is used to make better content recommendation decisions. The system collects user interaction data such as watch history, likes, dislikes, and content preferences to understand user interests.

It also checks whether the recommended content is available in the cache. If the content is available in the cache, it can be delivered quickly. If the content is not available, the system fetches it from the server.The Reinforcement Learning agent learns from user interactions and feedback to improve future recommendations. The main goal is to provide personalized content recommendations while reducing response latency and improving system performance.

The proposed system introduces a cache-aware content recommendation framework that combines user preferences, content information, and cache availability while making recommendation decisions. Unlike conventional recommendation systems that mainly focus on user behavior and content relevance, the proposed approach considers the current state of the content delivery system as an additional factor.

- The system collects information related to the user's current interests, previous interactions, content characteristics, and available cache information. These parameters are used to represent the current condition of the system and are provided to the reinforcement learning agent as its state.
- The user preference component represents the type of content that is relevant to the user based on previous interactions and current interests. This allows the system to maintain personalization while making recommendation decisions.
- The cache state represents the availability of relevant content in the cache. Information about whether particular content is cached or requires retrieval can be considered by the recommendation process. This helps the system identify recommendations that can potentially be delivered more efficiently.
- A Reinforcement Learning (RL) agent is used to make recommendation decisions. The agent observes the current state of the system and selects suitable content as an action. Instead of depending completely on fixed recommendation rules, the agent learns from the results of its previous decisions.
- After an action is selected, the system evaluates the recommendation using factors such as user preference, recommendation relevance, cache availability, user engagement, and content delivery efficiency. Based on these factors, a reward is provided to the reinforcement learning agent.
- The proposed system is also designed to handle changing user preferences. When the user's interests change over time, the observed state changes accordingly. The reinforcement learning agent can use this updated information to adjust its recommendation decisions instead of relying only on older interaction patterns.
- The system therefore treats recommendation and cache information as connected decision factors rather than completely independent processes. This allows the

recommendation strategy to consider whether content is relevant to the user as well as whether its availability can support efficient delivery.

- Over repeated interactions, the reinforcement learning agent learns a policy that determines which content should be recommended under different combinations of user preferences and cache conditions. The learned policy aims to achieve a balance between personalization and system performance.
- The proposed approach can help reduce unnecessary requests for content that is not readily available in the cache and encourage better utilization of cached resources. It can also help reduce content retrieval overhead and improve the efficiency of content delivery.
- The framework can be applied to different content-intensive environments, including video streaming platforms, news applications, music services, e-learning platforms, and Content Delivery Networks (CDNs), where both personalized recommendations and efficient content delivery are important.
- Overall, the proposed system combines personalized recommendation, cache awareness, and reinforcement learning into a unified framework. Its main objective is to dynamically balance recommendation relevance with cache and delivery efficiency while adapting to changing user preferences and system conditions.

4.2 Working of Proposed System

The proposed system works through the following steps:

1. **User Interaction:** The system collects user activities such as clicks, likes, dislikes, and watch history.
2. **Preference Analysis:** The collected data is analyzed to identify user interests and preferences.
3. **Cache State Checking:** The system checks whether the requested content is available in the cache.
4. **Reinforcement Learning Agent:** The agent considers user preferences and cache state to select suitable content.
5. **Content Recommendation:** The system ranks and recommends relevant content to the user.
6. **User Feedback:** The system collects feedback based on user interactions and uses it to improve future recommendations.

4.3 Advantages of Proposed System

- Personalized Recommendations: The system considers the user's preferences, previous interactions, and current interests to provide content that is more relevant to the individual user.
- Cache-Aware Recommendation: Unlike traditional recommendation systems, the proposed system considers the current cache availability while making recommendation decisions. This allows cache information to influence the selection of suitable content.

- Adaptive Decision-Making: The reinforcement learning agent learns from previous interactions and continuously improves its recommendation strategy based on the observed results.
- Adaptation to Changing User Preferences: The system can respond to changes in user interests by considering updated user behaviour and current preferences during the recommendation process.
- Improved Cache Utilization: By considering cached content during recommendation, the system can encourage better utilization of available cached resources instead of treating caching and recommendation as completely separate processes.
- Reduced Unnecessary Content Retrieval: Considering cache availability can help avoid situations where relevant content repeatedly needs to be retrieved from the origin server when suitable cached content is available.
- Reduced Network Latency: Making use of content that is already available closer to the user can potentially reduce the time required for content retrieval and improve delivery efficiency.
- Reduced Origin Server Workload: Better utilization of cached content can reduce repeated requests to the origin server, thereby helping to decrease server workload.
- Balanced Personalization and Performance: The system attempts to balance two important objectives: providing relevant recommendations to users and supporting efficient content delivery.

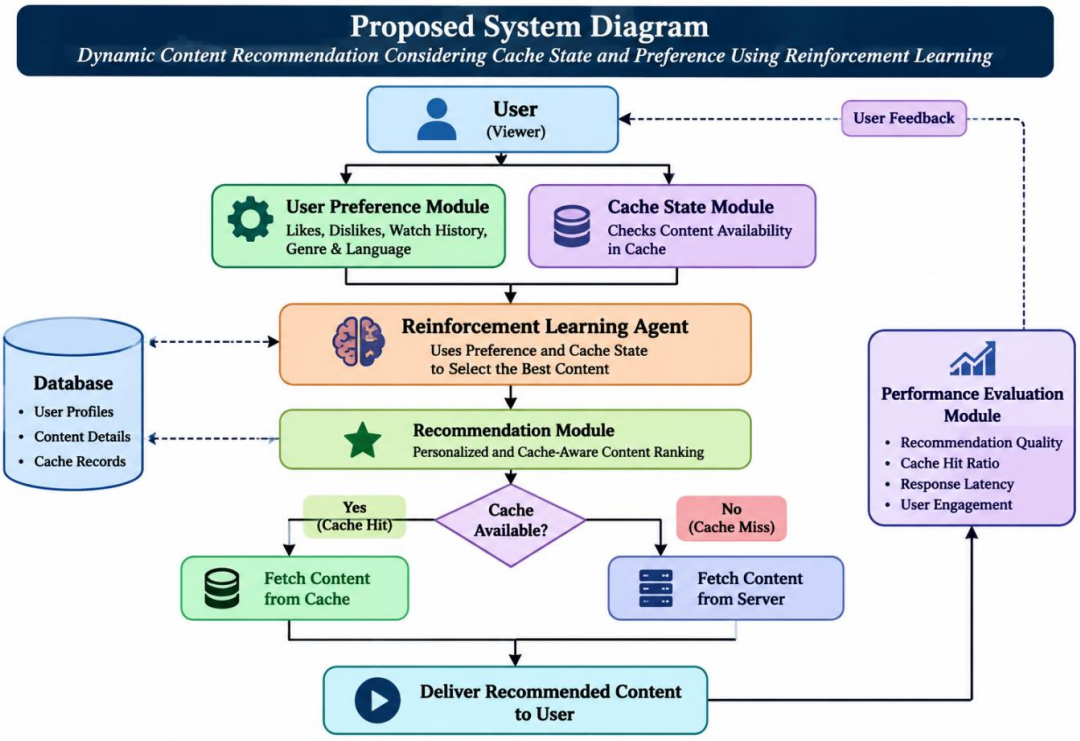


Figure 4: Proposed System

5.SYSTEM ARCHITECTURE

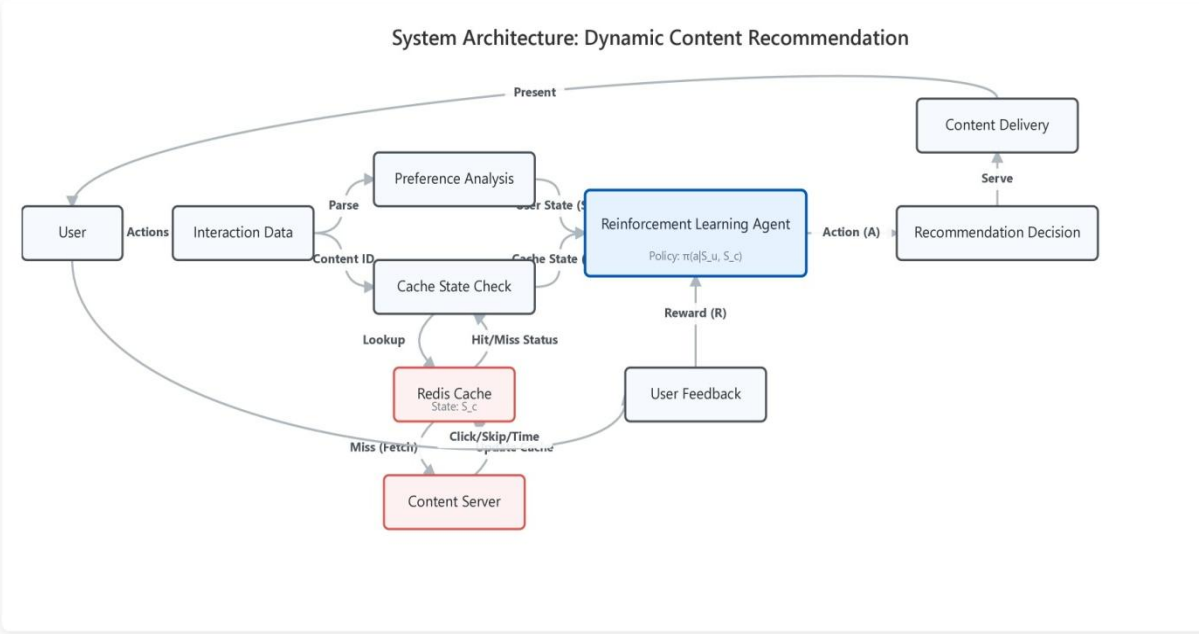


Figure 5: System Architecture

5.1 SYSTEM REQUIREMENTS

5.1.1 Hardware Requirements

The hardware requirements for the proposed system are:

- **System:** Intel Core i3 Processor
- **Hard Disk:** 1 TB
- **Monitor:** 14-inch Colour Monitor
- **Mouse:** Optical Mouse
- **RAM:** 4 GB

5.1.2 Software Requirements

The software requirements for the proposed system are:

- **Operating System:** Windows 10
- **Programming Language:** Python
- **Front-End:** HTML, CSS
- **Scripting Language:** JavaScript
- **Database:** SQLite
- **Development Environment:** Visual Studio Code

5.2 Description of System Architecture

The system architecture represents the overall structure and working process of the proposed content recommendation system.

1. User

The user interacts with the system by viewing content, clicking, liking, or disliking items.

2. User Interaction Data

The system collects user activities such as watch history, clicks, likes, dislikes, and viewing duration.

3. User Preference Module

This module analyzes user interaction data and identifies the user's interests, preferences, genres, and content choices.

4. Cache State Module

This module checks whether the required content is available in the cache. If the content is available, it is fetched from the cache. Otherwise, it is fetched from the server.

5. Reinforcement Learning Agent

The RL agent takes user preferences and cache information as input. It selects suitable recommendation actions and learns from user feedback using a reward mechanism.

6. Recommendation Module

This module ranks the selected content and provides personalized recommendations based on user preferences and cache availability.

7. User Feedback.

5.3 Working of System Architecture

The system starts by collecting user interaction data. The preference module analyzes this information to identify user interests. Next, the cache state module checks content availability.

The Reinforcement Learning agent considers the user preferences and cache state to make recommendation decisions. The recommendation module then provides suitable content to the user.

If content is available in the cache, it is fetched from the cache. Otherwise, it is retrieved from the server. Finally, user feedback is collected and used to improve future recommendations.

6.MODULES

6.1 Introduction

The proposed system, "Dynamic Content Recommendation Considering Cache State and Preference Using Reinforcement Learning", is divided into five main modules. Each module performs a specific task to make the content recommendation process more efficient and personalized.The main purpose of these modules is to analyze user preferences, check content availability in the cache, select suitable content using Reinforcement Learning, provide personalized recommendations, and evaluate the overall system performance.

1. User Preference Module
2. Cache State Module
3. Reinforcement Learning Module
4. Recommendation Module
5. Performance Evaluation Module

6.2 User Preference Module

The User Preference Module is responsible for collecting and analyzing user information to understand individual interests and content choices. This module plays an important role in providing personalized recommendations.The system collects different types of user interaction data, such as likes, dislikes, watch history, genre preferences, language preferences, clicks, and viewing duration. These details help the system understand what type of content the user is interested in.For example, if a user frequently watches comedy videos and likes Telugu-language content, the system identifies these interests and gives more importance to similar content during the recommendation process.

Functions of User Preference Module:

- Collects user likes and dislikes.
- Maintains user watch history.
- Identifies preferred content genres.
- Analyzes language preferences.
- Tracks clicks and viewing duration.
- Creates user preference information for recommendation decisions.

6.3 Cache State Module

The Cache State Module is responsible for checking whether the requested or recommended content is available in the cache memory. Cache memory stores frequently accessed content so that it can be retrieved quickly without repeatedly accessing the main server.When a user requests content, this module checks its availability in the cache. If the content is already available, the system retrieves it directly from the cache. This is called a cache hit.If the requested content is not.

Functions of Cache State Module:

- Checks content availability in cache memory.
- Identifies cache hits and cache misses.
- Retrieves available content from the cache.
- Fetches unavailable content from the server.
- Provides cache information to the Reinforcement Learning Module.
- Helps reduce response latency and server requests.

6.4 Reinforcement Learning Module

The Reinforcement Learning Module is the main decision-making component of the proposed system. It uses Reinforcement Learning techniques to select suitable content based on user preferences and cache availability.Reinforcement Learning is a machine learning approach in which an agent learns by performing actions and receiving rewards or penalties based on the results.In this project, the Reinforcement Learning agent receives information from the User Preference Module and the Cache State Module. Based on these inputs, the agent selects an appropriate recommendation action.

Functions of Reinforcement Learning Module:

- Receives user preference information.
- Considers cache availability information.
- Selects suitable recommendation actions.
- Learns from user interactions and feedback.
- Uses reward-based learning to improve decisions.
- Adapts recommendations based on previous results.
- Improves future recommendation quality.

6.5 Recommendation Module

The Recommendation Module is responsible for selecting, ranking, and displaying suitable content to users. It uses the decisions and information provided by the Reinforcement Learning Module.This module considers user interests, content relevance, and cache availability while preparing the recommendation list.The system gives importance to content that matches the user's preferences. It also considers whether the selected content is available in the cache to support faster content delivery..

6.6 Performance Evaluation Module

The Performance Evaluation Module is responsible for measuring and analyzing the overall performance of the proposed system. It helps determine whether the recommendation process is working effectively.This module evaluates the system using different performance metrics, including recommendation quality, cache hit ratio, response latency, bandwidth usage, and user engagement.Recommendation quality measures how relevant the suggested content is to the user's interests.Cache hit

7.UML DIAGRAMS

Unified Modeling Language (UML) is a standard visual modeling language used to represent the design and behavior of a software system. UML diagrams help developers and readers understand the system structure, user interactions, workflow, and communication between different components. The proposed Dynamic Content Recommendation Considering Cache State and Preference Using Reinforcement Learning system is represented using four UML-style diagrams: Use Case Diagram, Activity Diagram, Sequence Diagram, and Class Diagram. These diagrams describe the major functions and interactions of the proposed system.

7.1 Use Case Diagram

The Use Case Diagram represents the interactions between the user and the content recommendation system. The user can access the system, view content, provide likes or dislikes, and view watch history. The system analyzes preferences, checks cache availability, generates recommendations, and delivers content. This diagram gives a high-level view of the services provided by the system.

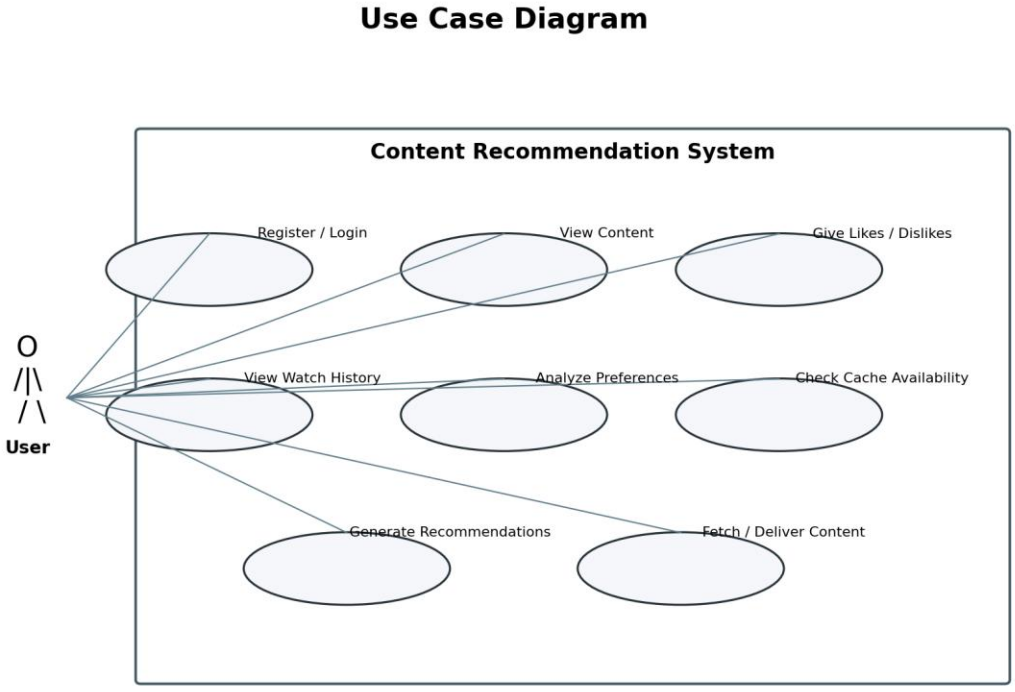


Figure 7.1: Use Case Diagram

7.2 Activity Diagram

The Activity Diagram explains the step-by-step workflow of the proposed system. The process starts when the user accesses the system. User interaction data is collected and analyzed to identify preferences. The cache state module checks content availability, and the preference and cache information are supplied to the.

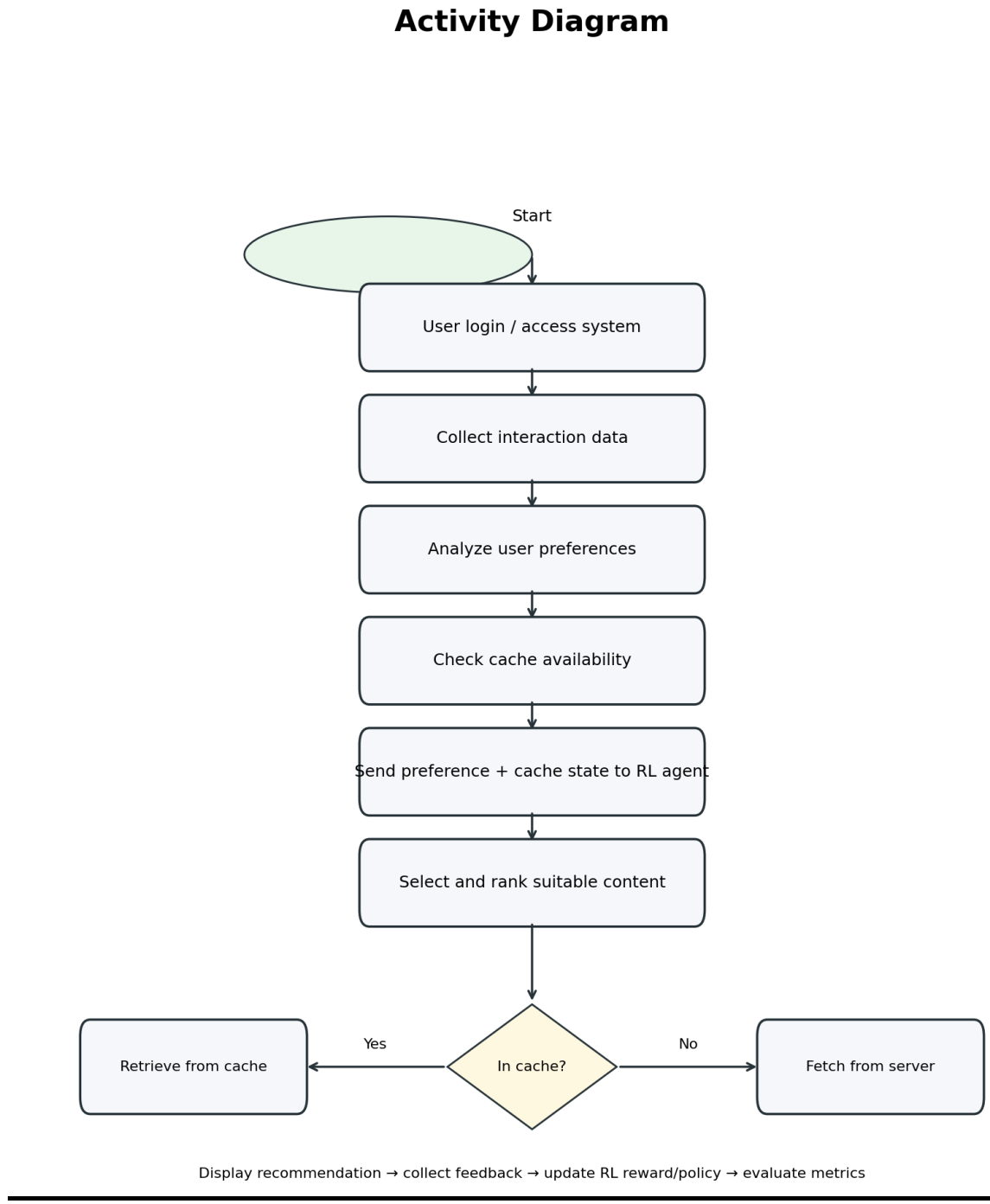


Figure 7.2: Activity Diagram

7.3 Sequence Diagram

The Sequence Diagram shows the order of communication among the user and the major system modules. The user submits interaction data to the User Preference Module. The preference information and cache state are passed to the Reinforcement Learning agent. The agent selects a recommendation and sends it to the Recommendation Module. The content is retrieved through the Cache State Module and displayed to the user. User feedback is returned to the learning agent, while the Performance Evaluation Module measures system metrics.

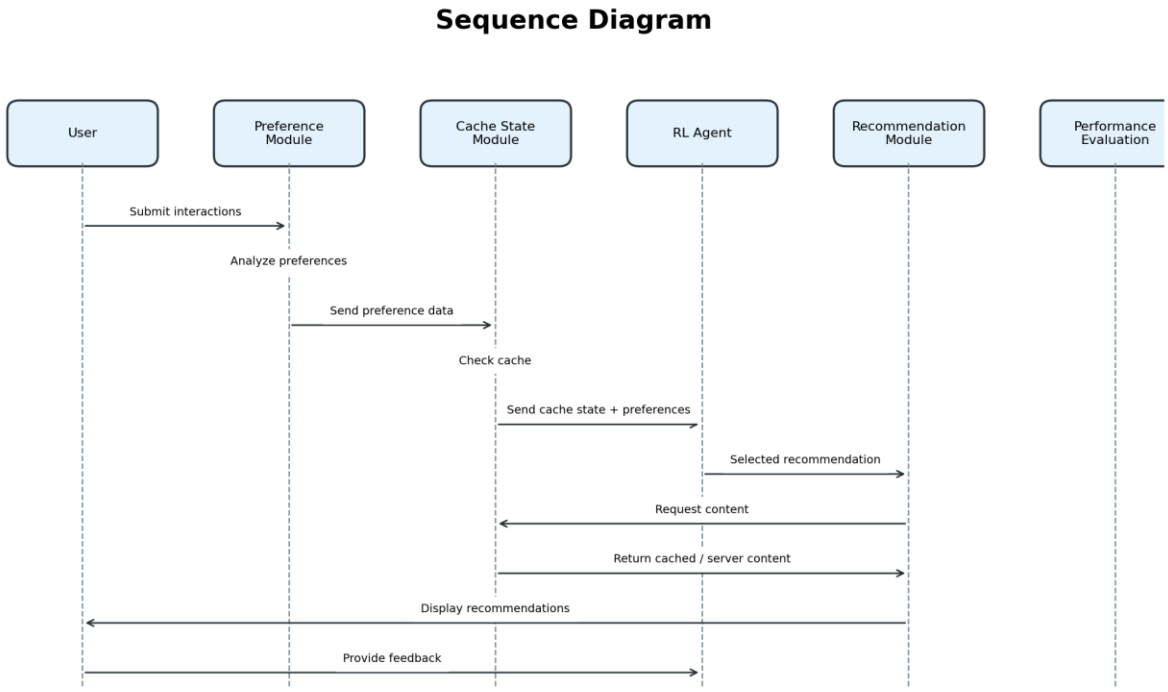


Figure 7.3: Sequence Diagram

7.4 Class Diagram

The Class Diagram describes the static structure of the system through classes, attributes, operations, and relationships. The User class represents the person interacting with the application. UserPreference stores preference-related information. CacheState checks whether content is available in cache.

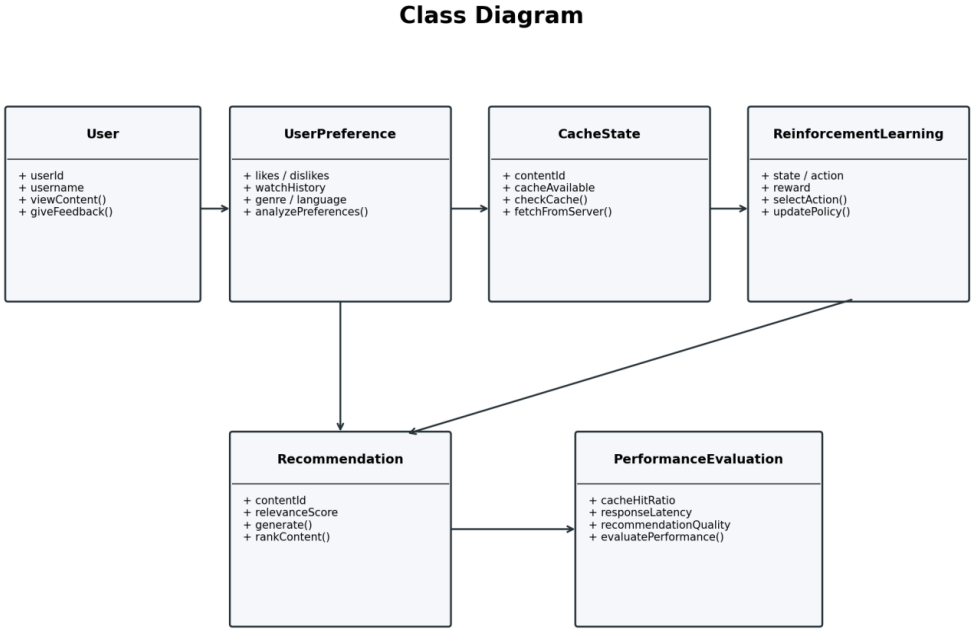


Figure 7.4: Class Diagram

7.5 DATA FLOW DIAGRAM (DFD)

7.5.1 Introduction

A Data Flow Diagram (DFD) is used to represent how data moves through a system. It explains how information is received from users, processed by different modules, stored, and delivered as output.In our project, Dynamic Content Recommendation Considering Cache State and Preference Using Reinforcement Learning, the DFD explains the flow of user interaction data, user preferences, cache information, reinforcement learning operations, and recommended content.The system collects user activities such as likes, dislikes, watch history, and content requests. Based on this information, it analyses user preferences and checks the cache availability of requested content. The Reinforcement Learning agent uses these details to select suitable content and improve recommendations using user feedback.

7.5.2 DFD Level 0 – Context Diagram

Working:

1. The user sends content requests and provides interaction details to the system.
2. The system receives user preferences such as likes, dislikes, watch history, and preferred genres.
3. The system checks whether the requested content is available in the cache.
4. If the content is available in the cache, it retrieves the content from the cache.
5. If the content is not available, the system fetches the content from the content server.
6. The system uses the Reinforcement Learning agent to select suitable content.
7. Finally, personalized recommended content is delivered to the user.

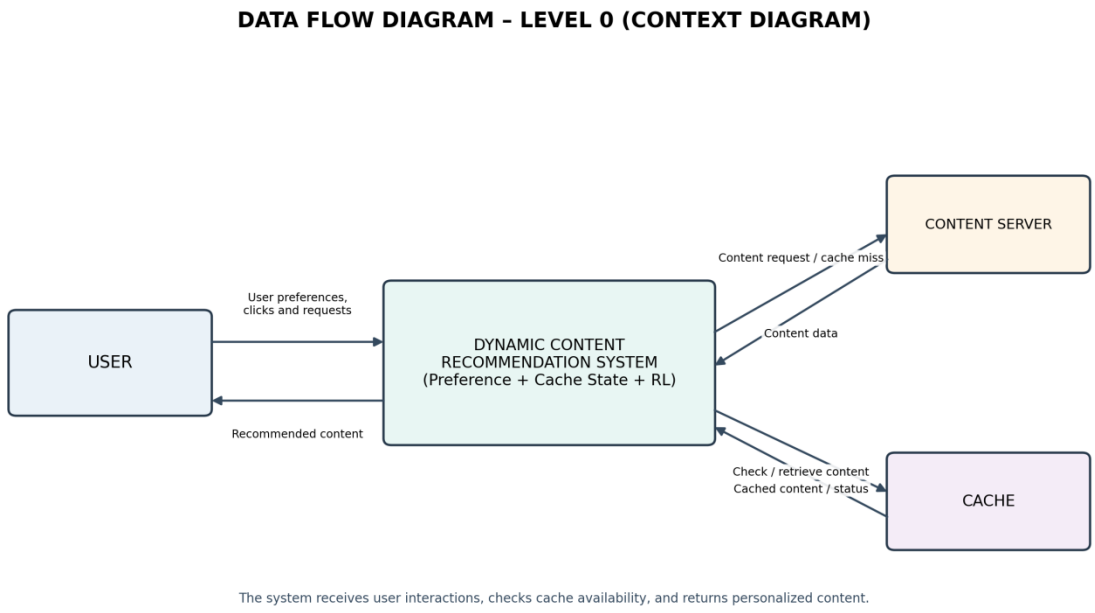


Figure 7.5.2: DFD Level 0 – Context Diagram

7.5.3 DFD Level 1

The Level 1 Data Flow Diagram provides a detailed view of the internal processes of the recommendation system. It divides the complete system into different processes and explains how data moves between them.

1. User Input and Preference Analysis

This process collects user interaction information, including watch history, likes, dislikes, genre preferences, language preferences, and content requests. The collected information is analysed to understand the user's interests.

2. Cache State and Reinforcement Learning Processing

This process checks whether the requested content is available in the cache. The cache status is provided to the Reinforcement Learning agent along with user preference information. The agent uses these details to select suitable content.

3. Recommendation and Content Delivery

This process generates personalized and cache-aware content recommendations. If the content is available in the cache, it is retrieved from the cache. Otherwise, the content is fetched from the content server. The recommended content is then delivered to the user.

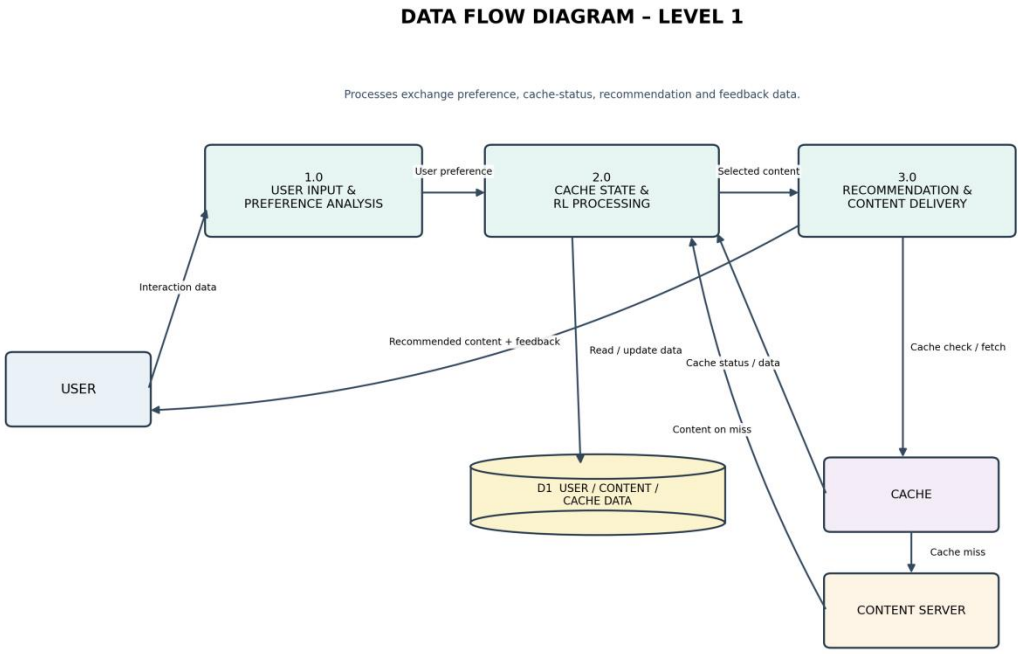


Figure 7.5.3: DFD Level 1

7.6 DEPLOYMENT DIAGRAM

7.6.1 Introduction

A UML Deployment Diagram is used to represent the physical deployment of a software system. It explains how software components are arranged on hardware devices or servers and how these components communicate with each other.In our project, the Deployment Diagram represents the arrangement of the user device, application server, cache server, and content server.

7.6.2 Working of Deployment Diagram

1. The user accesses the application through a web browser.
2. The browser sends the user's request to the application server.
3. The application server processes the request.
4. The recommendation engine analyses user preferences and checks cache status.
5. The system communicates with the cache server to check content availability.
6. If the content is available, it is retrieved from the cache server.
7. If the content is not available, the application server requests it from the content server.
8. The recommendation system selects suitable content using the Reinforcement Learning agent.
9. The recommended content is delivered to the user through the web browser.
10. User feedback is used to improve future recommendations.

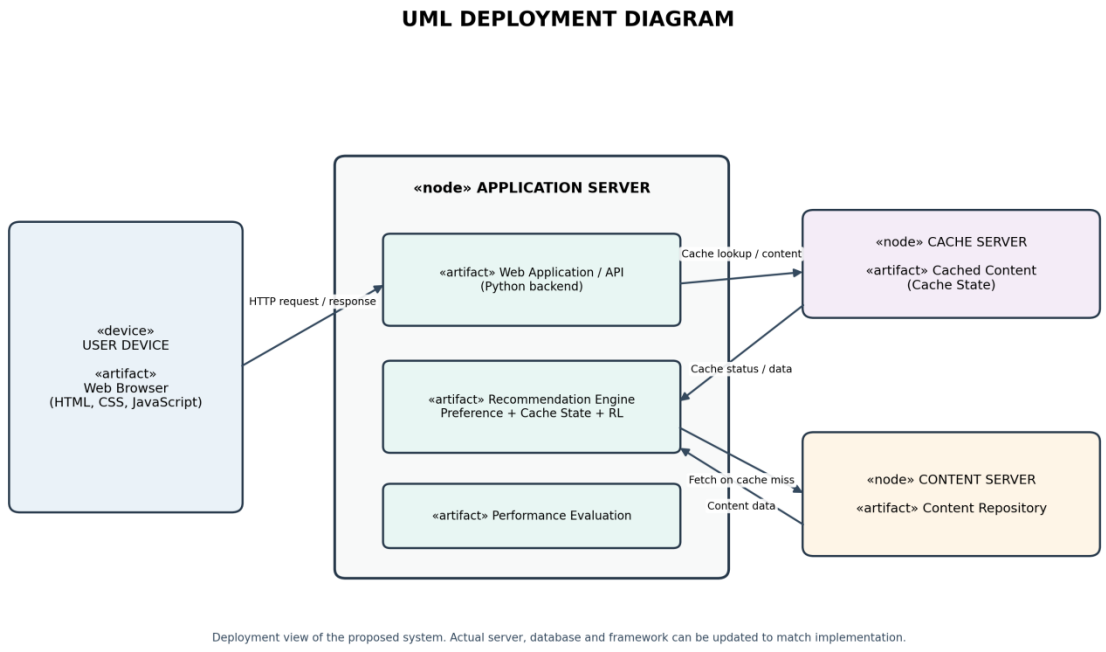


Figure 7.6: UML Deployment Diagram

8.REFERENCES

8.1 Research Papers – Recommendation Systems

1. Paul Resnick and Hal R. Varian, "Recommender Systems," Communications of the ACM, Vol. 40, No. 3, pp. 56–58, 1997.

2. B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-Based Collaborative Filtering Recommendation Algorithms," Proceedings of the 10th International Conference on World Wide Web, pp. 285–295, 2001.

3. Y. Koren, R. Bell, and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," Computer, Vol. 42, No. 8, pp. 30–37, 2009.

4. F. Ricci, L. Rokach, and B. Shapira, "Introduction to Recommender Systems Handbook," Recommender Systems Handbook, Springer, 2011.

5. J. B. Schafer, D. Frankowski, J. L. Herlocker, and S. Sen, "Collaborative Filtering Recommender Systems," The Adaptive Web, Springer, pp. 291–324, 2007.

8.2 Research Papers – Reinforcement Learning

6. R. S. Sutton and A. G. Barto, "Reinforcement Learning: An Introduction," MIT Press, 2018.

7. C. J. C. H. Watkins and P. Dayan, "Q-Learning," Machine Learning, Vol. 8, pp. 279–292, 1992.

8. V. Mnih et al., "Human-Level Control Through Deep Reinforcement Learning," Nature, Vol. 518, pp. 529–533, 2015.

9. J. Li, "Reinforcement Learning for Recommender Systems," Proceedings of the 14th ACM Conference on Recommender Systems, 2020.

8.3 Research Papers – Cache Management and Content Delivery

10. J. O. Kephart and D. M. Chess, "The Vision of Autonomic Computing," Computer, IEEE, Vol. 36, No. 1, pp. 41–50, 2003.

11. M. Podlipnig and L. Böszörmenyi, "A Survey of Web Cache Replacement Strategies," ACM Computing Surveys, Vol. 35, No. 4, pp. 374–398, 2003.

12. J. G. S. A. D. Almeida et al., "A Survey on Web Caching," ACM Computing Surveys.

13. M. S. Arlitt and C. L. Williamson, "Web Server Workload Characterization: The Search for Invariants," ACM SIGMETRICS Performance Evaluation Review, 1996.

8.4 Books

14. Richard S. Sutton and Andrew G. Barto, Reinforcement Learning: An Introduction, MIT Press, 2018.

15. Charu C. Aggarwal, Recommender Systems: The Textbook, Springer, 2016.

16. Christopher M. Bishop, Pattern Recognition and Machine Learning, Springer, 2006.

8.5 Programming Languages and Technologies

18. Python Software Foundation, “Python Documentation.” Available at: https://docs.python.org/3/

19. SQLite, “SQLite Documentation.” Available at: https://www.sqlite.org/docs.html

20. MDN Web Docs, “HTML: HyperText Markup Language.” Available at: https://developer.mozilla.org/en-US/docs/Web/HTML

22. MDN Web Docs, “JavaScript Guide.” Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

8.6 Online Research Platforms

24. IEEE Xplore Digital Library – https://ieeexplore.ieee.org/

25. Google Scholar – https://scholar.google.com/

26. ScienceDirect – https://www.sciencedirect.com/

27. SpringerLink – https://link.springer.com/

28. ACM Digital Library – https://dl.acm.org/

29. ResearchGate – https://www.researchgate.net/

8.7 Official Documentation

30. Python Software Foundation – Python programming documentation.

31. SQLite – Database management documentation.

32. MDN Web Docs – Frontend development documentation.

33. IEEE – Research publications related to computer science and engineering.

arXiv:2404.14961v1 [cs.LG] 23 Apr 2024

Cache-Aware Reinforcement Learning in Large-Scale Recommender Systems

Xiaoshuang Chen*
Kuaishou Technology
Beijing, China
chenxiaoshuang@kuaishou.com

Gengrui Zhang*
Kuaishou Technology
Beijing, China
zhanggengrui@kuaishou.com

Yao Wang
Kuaishou Technology
Beijing, China
wangyiyan@kuaishou.com

Yulin Wu
Kuaishou Technology
Beijing, China
wuyulin@kuaishou.com

Shuo Su
Kuaishou Technology
Beijing, China
sushuo@kuaishou.com

Kaiqiao Zhan
Kuaishou Technology
Beijing, China
zhankaiqiao@kuaishou.com

Ben Wang†
Kuaishou Technology
Beijing, China
wangben@kuaishou.com

ABSTRACT

Modern large-scale recommender systems are built upon computation-intensive infrastructure and usually suffer from a huge difference in traffic between peak and off-peak periods. In peak periods, it is challenging to perform real-time computation for each request due to the limited budget of computational resources. The recommendation with a cache is a solution to this problem, where a user-wise result cache is used to provide recommendations when the recommender system cannot afford a real-time computation. However, the cached recommendations are usually suboptimal compared to real-time computation, and it is challenging to determine the items in the cache for each user. In this paper, we provide a cache-aware reinforcement learning (CARL) method to jointly optimize the recommendation by real-time computation and by the cache. We formulate the problem as a Markov decision process with user states and a cache state, where the cache state represents whether the recommender system performs recommendations by real-time computation or by the cache. The computational load of the recommender system determines the cache state. We perform reinforcement learning based on such a model to improve user engagement over multiple requests. Moreover, we show that the cache will introduce a challenge called critic dependency, which deteriorates the performance of reinforcement learning. To tackle this challenge, we propose an eigenfunction learning (EL) method to learn independent critics for CARL. Experiments show that CARL can significantly improve the users' engagement when considering the result cache. CARL has been fully launched in Kwai app, serving over 100 million users.

CCS CONCEPTS

• **Information systems** → **Recommender systems**.

KEYWORDS

Reinforcement Learning, Recommender Systems, Cache, Eigenfunction

ACM Reference Format:
Xiaoshuang Chen, Gengrui Zhang, Yao Wang, Yulin Wu, Shuo Su, Kaiqiao Zhan, and Ben Wang. 2024. Cache-Aware Reinforcement Learning in Large-Scale Recommender Systems. In *Companion Proceedings of the ACM Web Conference 2024 (WWW '24 Companion), May 13–17, 2024, Singapore, Singapore.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3589335.3648326

1 INTRODUCTION

Recently, reinforcement learning (RL) has drawn a growing attraction in recommender systems [1, 2, 4, 13, 15, 16]. RL-based recommendation methods treat users as the environment and the recommender system as the agent and then model the sequential interactions between users and the recommender system. RL achieves success in improving users' long-term engagements, such as the total dwell time [2, 15] and the user retention [1].

Existing RL methods require the system to interact with the users and change the next recommendation according to the user feedback in real-time. However, modern large-scale recommender systems are built upon computation-intensive infrastructure [12]. Although there is an amount of research on reducing the computational burden of recommender systems [6, 7, 9], it is still challenging to perform real-time computation for each request during peak periods. Fig. 1 shows the queries-per-second (QPS) of Kwai app based on the average QPS per day. It is shown that the computational burden during peak periods is several times that of off-peak periods. It

*Both authors contributed equally to this research.
†Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
WWW '24 Companion, May 13–17, 2024, Singapore, Singapore
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0172-6/24/05…$15.00
https://doi.org/10.1145/3589335.3648326

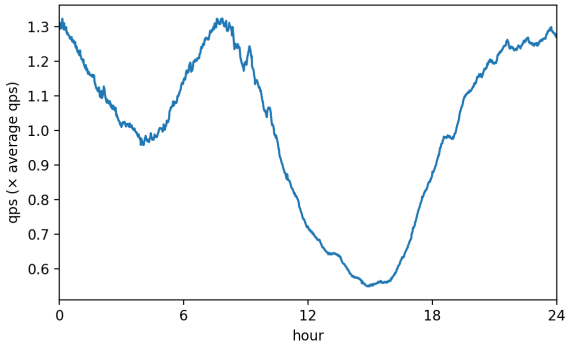


Figure 1: QPS in one day of Kwai app.

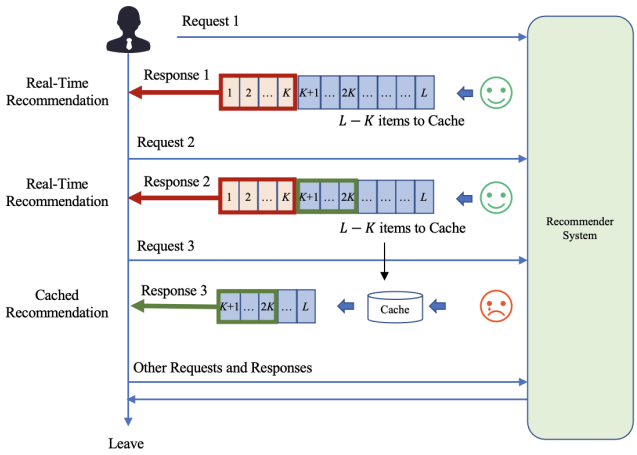


Figure 2: Recommendation with a result cache.

requires a lot of computational resources to perform real-time computation in the peak period, while the same resources will be idle in the off-peak periods, which is not cost-effective. Therefore, a result cache [5, 11] is used to balance the computational burden and the recommendation performance in large-scale recommender systems. Fig. 2 briefly shows the recommender system with a result cache in Kwai. When a user sends a request, the system first provides a real-time recommendation, which returns a group of L items, of which the top K items are showed to the user while the other $L-K$ are put into a result cache. When the user's next request comes, the recommender system will perform a real-time computation if the traffic does not exceed the system's affordability, e.g. Response 2 in Fig. 2. Otherwise, it will recommend K items directly from the result cache, e.g. Response 3 in Fig 2. The existence of the cache mitigates the computational burden of the recommender systems in peak periods, but it brings several challenges to traditional RL approaches:

- **Lack of actions in the cache**. The RL module is usually a part of the real-time computation, meaning RL will not be performed when the request is processed by the cache. This contradicts the assumption of RL to interact with the users continuously.
- **Unpredictability of the cache choice**. The choice of using real-time computation or the cache depends on the total computational burden of the system rather than the features of the specific request. Therefore, the cache choice is unpredictable by the RL algorithm, which increases the learning difficulty.
- **Heterogenuous rewards**. the performance of cached results is suboptimal compared to the results from the real-time computation due to the lack of instant behaviors of the user. Table 1 provides an example in the Kwai app, where the average user engagement of cached recommendations is lower than that of real-time recommendations. Such characteristics need to be considered in the RL model.

To this end, we provide a novel cache-aware reinforcement learning (CARL) method. CARL introduces a cache state to model the choice of recommendations by real-time computation or by the cache. The cache state is determined by the computational burden of the recommender system and is independent of the user states

	Real-Time Recommendations	Cached Recommendations
Watch Time	×1	×0.85
Like Rate	×1	×0.68
Follow Rate	×1	×0.54

Table 1: Comparisons between User engagement in real-time and cached recommendations in Kwai.

and actions of the system. We also show that the existence of the cache introduces challenges to CARL training. Specifically, the uncontrollable cache state introduces an uncontrollable dependency between the critic functions of real-time cases and cached cases, which deteriorates the performance of the training algorithm. To tackle this challenge, we introduce a novel eigenfunction learning (EL) technique to train CARL. EL learns two independent critics and then combines them to obtain the critic functions of real-time and cached recommendations, improving CARL's performance.

In summary, our contributions are as follows:

- We introduce a novel CARL method to model the recommender systems with cache explicitly.
- We show that the existence of the cache introduces critic dependency to the RL algorithm, which deteriorates the performance. Then, we provide an EL algorithm to train CARL effectively.
- Experiments show that CARL improves users' engagement in recommender systems with the cache. CARL has been launched in Kwai app, serving over 100 millions of users.

Following the introduction, Section 2 provides the CARL model to describe the recommendation process with a cache. Section 3 discusses the challenges of learning CARL, and provides an effective EL algorithm to train CARL. Section 4 provides the experimental results, and Section 5 concludes the paper.

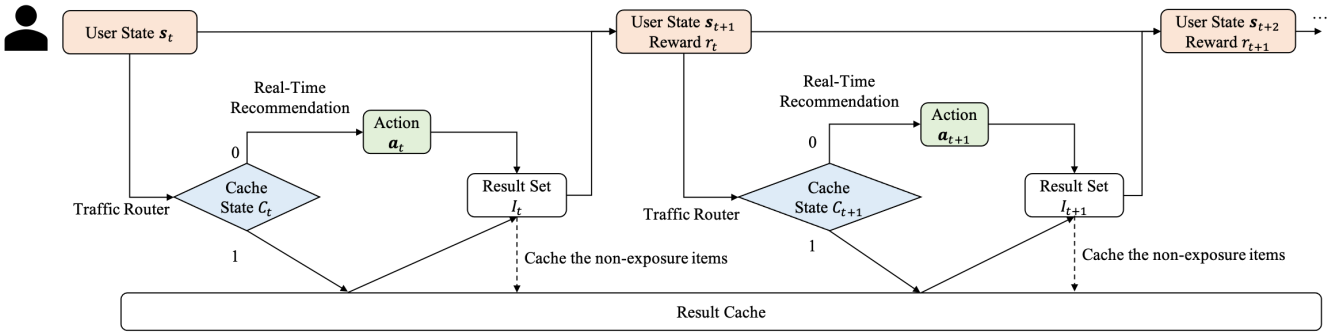


Figure 3: The CARL model.

2 MODELING OF CARL

This section provides the CARL model, which is an RL model considering the existence of the result cache, as shown in Figure 3. We model the interaction between users and the recommender systems as a Markov Decision Process (MDP), where the recommender system is the agent, and users are the environments. When a user opens the app, a session begins, which consists of multiple requests until the user leaves the app. At Step t, the recommender system obtains a user state $\boldsymbol{s_t}$. The user state $\boldsymbol{s_t}$ consists of the user profile, behavior history, request context, and candidate video features. According to the user request with the state $\boldsymbol{s_t}$, the recommender system generates an action $\boldsymbol{a_t}$, and outputs a set of items I_t to the user according to the user state $\boldsymbol{s_t}$ and the action $\boldsymbol{a_t}$. After watching the provided items in I_t, the user provides feedback r_t, e.g. the watching time of the items and other interactions on the items. Then, the user transfers to the next state $\boldsymbol{s_{t+1}}$ and determines whether to send the next request or leave. The recommender system aims to maximize the long-term reward R_t defined by

$$R_t = \sum_{t'=t}^{T} \gamma^{t'-1} \mathbb{E}\left[r_{t'}\right], \tag{1}$$

where T is the last step and γ is the discount factor.

A key component to be modeled in the recommender systems with cache is the choice of real-time computation or cached results. Specifically, the system maintains a result cache $\mathcal{K}_u$ for each user u. When a user request comes at Step t, a traffic router will provide a cache state $C_t \in \{0, 1\}$, and the system will provide different services according to C_t:

- **Recommendation by real-time computation**. When the recommender system has enough computational resources, the traffic router returns $C_t = 0$, and the recommender system will use a real-time computation to provide recommended items. Specifically, n deep models will be used to predict scores $\boldsymbol{x}_{j,t} \in \mathbb{R}^n$ for each candidate item j, in terms of various feedback (watch time, follow, like, etc.). Then, the action $\boldsymbol{a_t}$ is used to generate the final score of each item according to a parametrized ranking function f, i.e.

$$z_{j,t} = f(\boldsymbol{x}_{j,t}; \boldsymbol{a_t}) \tag{2}$$

  Finally, the top L items regarding the scores $z_{j,t}$ are used, of which the top K items are recommended to the user, while the items with rank $K+1$ to L are put into the result cache $\mathcal{K}_u$ for future use.
- **Recommendation by the cache**. When the traffic of the recommender system is too large to afford a real-time computation, the traffic router returns $C_t = 1$, and the system will provide recommendations by the cache. In such cases, the recommender system obtains K items directly from the user's result cache $\mathcal{K}_u$. Then, $\mathcal{K}_u$ is updated by removing the K items recommended. There is no action $\boldsymbol{a_t}$ in recommendations by the cache.

In our scenario, the traffic router maintains a queue of ongoing real-time recommendation processes. When a new request comes, the traffic router checks the queue length. If the queue length does not exceed a given limit, it suggests a real-time computation; otherwise, it suggests a cached recommendation.

Given the abovementioned models, CARL aims to maximize the aggregate reward R_t defined in Eq. (1). The main challenge of training CARL arises from the stochastic characteristic of the traffic router. The cache state C_t depends on the total traffic of the system rather than the user state $\boldsymbol{s_t}$. Therefore, C_t is unpredictable according to $\boldsymbol{s_t}$. Such stochastic characteristics make it challenging to learn the critic. Section 3 will discuss the challenge in detail and provide an effective training method.

3 LEARNING OF CARL

This section first shows the application of traditional RL algorithms to CARL, and discusses the critic dependency problem of the direct learning approach. Then, we provide the EL algorithm to learn CARL effectively.

3.1 Challenges of Direct Learning

3.1.1 The Direct Learning Algorithm. We first show a direct application of traditional RL to learn the CARL model. We consider a typical actor-critic architecture. The action $\boldsymbol{a_t}$ is determined by a policy function μ with the parameter θ:

$$\boldsymbol{a_t} = \mu(\boldsymbol{s_t}; \theta) \tag{3}$$

We use a critic function $Q(\boldsymbol{s_t}, \boldsymbol{a_t}; \phi)$, parameterized by ϕ to estimate the long-term reward R_t given the state $\boldsymbol{s_t}$ and the action $\boldsymbol{a_t}$. A typical critic learning algorithm [8] uses a temporal difference

	$C_t = 0$	$C_t = 1$
$C_{t+1} = 0$	$\left[Q_0\left(\boldsymbol{s}_t, \boldsymbol{a}_t; \phi_0\right) - \left(r_t + \gamma Q_0\left(\boldsymbol{s}_{t+1}, \mu\left(\boldsymbol{s}_{t+1}; \theta^-\right); \phi_0^-\right)\right)\right]^2$	$\left[Q_1\left(\boldsymbol{s}_t; \phi_1\right) - \left(r_t + \gamma Q_0\left(\boldsymbol{s}_{t+1}, \mu\left(\boldsymbol{s}_{t+1}; \theta^-\right); \phi_0^-\right)\right)\right]^2$
$C_{t+1} = 1$	$\left[Q_0\left(\boldsymbol{s}_t, \boldsymbol{a}_t; \phi_0\right) - \left(r_t + \gamma Q_1\left(\boldsymbol{s}_{t+1}, ; \phi_1^-\right)\right)\right]^2$	$\left[Q_1\left(\boldsymbol{s}_t, \boldsymbol{a}_t; \phi_1\right) - \left(r_t + \gamma Q_1\left(\boldsymbol{s}_{t+1}; \phi_1^-\right)\right)\right]^2$

Table 2: Difference formulations of $l_{DL}(\phi)$ in Eq. (8)

.

method to learn the parameter ϕ of the critic Q:

$$l\left(\phi\right) = \left[Q\left(\boldsymbol{s}_t, \boldsymbol{a}_t; \phi\right) - \left(r_t + \gamma Q\left(\boldsymbol{s}_{t+1}, \mu\left(\boldsymbol{s}_{t+1}; \theta^-\right); \phi^-\right)\right)\right]^2 \quad (4)$$

where θ^- is the parameter of the target actor, and ϕ^- is the parameter of the target critic. The critic function Q is used as the reward function of the policy function μ, and the parameter θ of the actor updates according to the following gradient ascent:

$$\nabla_\theta J = \nabla_\theta \mu(\boldsymbol{s}_t; \theta) \nabla_\mu Q\left(\boldsymbol{s}_t, \mu(\boldsymbol{s}_t; \theta)\right) \quad (5)$$

To apply Eq. (4)(5) to CARL, we define the conditional critic functions:

$$Q_0\left(\boldsymbol{s}_t, \boldsymbol{a}_t; \phi_0\right) \triangleq \mathbb{E}\left[R_t | C_t = 0\right], Q_1\left(\boldsymbol{s}_t; \phi_1\right) \triangleq \mathbb{E}\left[R_t | C_t = 1\right] \quad (6)$$

Q_0 and Q_1 are the expected long-term rewards under the condition that the Request t is processed by real-time computation and by the cache, respectively. The input of Q_1 does not contain the action $\boldsymbol{a}_t$ because there is no action in recommendations by the cache.

Given Q_0 and Q_1, the total critic function Q can be written as

$$Q(\boldsymbol{s}_t, \boldsymbol{a}_t; \phi) = Q_{C_t}(\boldsymbol{s}_t, \boldsymbol{a}_t; \phi_{C_t}), C_t \in \{0, 1\} \quad (7)$$

where $\phi = \{\phi_0, \phi_1\}$. With the critic defined in Eq. (7), we can directly train the CARL by the following critic loss:

CARL-DL (Direct Learning of CARL):

$$l_{DL}\left(\phi\right) = \left[Q_{C_t}\left(\boldsymbol{s}_t, \boldsymbol{a}_t; \phi\right) - \left(r_t + \gamma Q_{C_{t+1}}\left(\boldsymbol{s}_{t+1}, \mu\left(\boldsymbol{s}_{t+1}; \theta^-\right); \phi^-\right)\right)\right]^2, \quad (8)$$

with the policy loss remains the same as Eq. (5).

3.1.2 Critic Dependency. The CARL-DL approach is straightforward, but it faces a considerable challenge, i.e. the **Critic Dependency**. Specifically, Eq. (8) shows the dependency of $Q(\boldsymbol{s}_t, \boldsymbol{a}_t; \phi)$ and $Q(\boldsymbol{s}_{t+1}, \boldsymbol{a}_{t+1}; \phi)$. According to different cache states C_t and C_{t+1}, there are four possible formulations of Eq. (8), as shown in Table 2. It is clear that the Q_0 and Q_1 depend on each other. Figure 4 also shows the dependency among the two critics Q_0 and Q_1 in the DL of CARL. The critic dependency makes the learning process depending on the values of the cache states C_t and C_{t+1}, but the cache states are highly stochastic, and are unpredictable by the states $\boldsymbol{s}_t$ and the actions $\boldsymbol{a}_t$. Such problem deteriorates the convergence of the critic learning.

To discuss this problem more precisely, we consider the transit function of the critics. Since the cache state C_t is a stochastic variable, we use $D_c(t)$ to denote the probability that $C_t = c$:

$$D_c(t) \triangleq P(C_t = c), c \in \{0, 1\} \quad (9)$$

$D_c(t)$ does not depend on the user's state $\boldsymbol{s}_t$ because it simply relies on the total requests per second and the maximal real-time recommendations per second.

In addition, we define the expectation of the immediate reward r_t of different cache states C_t:

$$V_0\left(\boldsymbol{s}_t, \boldsymbol{a}_t\right) \triangleq \mathbb{E}\left[r_t | C_t = 0\right], V_1\left(\boldsymbol{s}_t\right) \triangleq \mathbb{E}\left[r_t | C_t = 1\right] \quad (10)$$

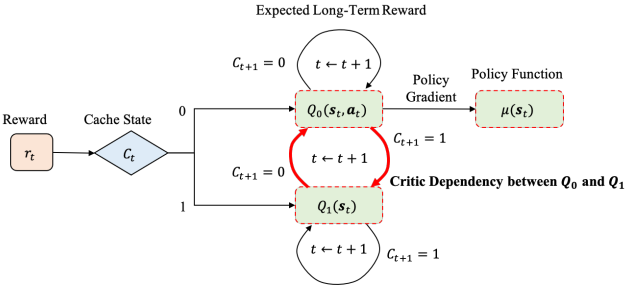


Figure 4: Direct Learning of CARL

Then, according to the backward induction, the transit functions of the expected immediate reward and the expected long-term reward can be written as:

$$\begin{aligned} Q_0\left(\boldsymbol{s}_t, \boldsymbol{a}_t\right) =& V_0\left(\boldsymbol{s}_t, \boldsymbol{a}_t\right) + \gamma D_0\left(t+1\right) \mathbb{E}_{\boldsymbol{a}_{t+1}} Q_0\left(\boldsymbol{s}_{t+1}, \boldsymbol{a}_{t+1}\right) \\ &+ \gamma D_1\left(t+1\right) \mathbb{E}_{\boldsymbol{a}_{t+1}} Q_1\left(\boldsymbol{s}_{t+1}\right) \\ Q_1\left(\boldsymbol{s}_t\right) =& V_1\left(\boldsymbol{s}_t\right) + \gamma D_0\left(t+1\right) \mathbb{E}_{\boldsymbol{a}_{t+1}} Q_0\left(\boldsymbol{s}_{t+1}, \boldsymbol{a}_{t+1}\right) \\ &+ \gamma D_1\left(t+1\right) \mathbb{E}_{\boldsymbol{a}_{t+1}} Q_1\left(\boldsymbol{s}_{t+1}\right) \end{aligned} \quad (11)$$

Now, we provide more explanations on Eq. (11), as shown in Figure 4. On the one hand, the critic functions Q_0 and Q_1, i.e. the expectation of the long-term reward R_t, depends on the immediate reward r_t, estimated by V_0 and V_1. On the other hand, Q_0 and Q_1 also depend on the future feedback of the user. However, due to the stochastic characteristic of the cache state C_t, the future feedback of the user obeys different distributions given different C_t, and the expectation of the future feedback is described by the second and third term of the right part of Eq. (11).

Eq. (11) also shows the critic dependency problem, since the Q_0 and Q_1 clearly depend on each other. Such dependency, together with the stochasticity of the cache states C_t, will increase the learning difficulty of CARL. We will discuss the solution to the critic dependency problem in the following subsections.

3.2 Eigenfunctions

This paper uses the eigenfunction technique to solve the critic dependency problem. The motivation is to find a group of independent variables derived from the critic functions Q_0 and Q_1 so that the learning process of different critic functions can be decoupled.

We first define the eigen immediate reward and the eigen long-term reward.

Definition 3.1 (Eigen Immediate Reward). The eigen immediate rewards, denoted by Γ_a and Γ_b, are given by

$$\begin{aligned} \Gamma_a\left(\boldsymbol{s}_t, \boldsymbol{a}_t\right) &\triangleq V_0\left(\boldsymbol{s}_t, \boldsymbol{a}_t\right) - V_1\left(\boldsymbol{s}_t\right) \\ \Gamma_b\left(\boldsymbol{s}_t, \boldsymbol{a}_t\right) &\triangleq D_0(t) V_0\left(\boldsymbol{s}_t, \boldsymbol{a}_t\right) + D_1(t) V_1\left(\boldsymbol{s}_t\right) \end{aligned} \quad (12)$$

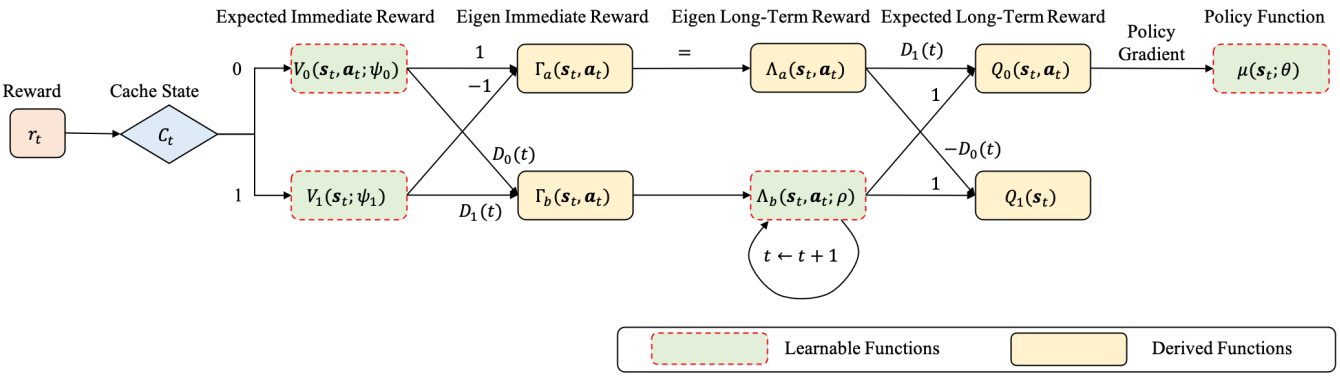


Figure 5: Eigenfunction Learning of CARL

The eigen immediate rewards Γ_a and Γ_b are the linear combinations of the expected immediate rewards V_0 and V_1. Similarly, we have

Definition 3.2 (Eigen Long-Term Reward). The eigen long-term rewards, denoted by Λ_a and Λ_b, are given by

$$\begin{aligned} \Lambda_a(\boldsymbol{s}_t, \boldsymbol{a}_t) &\triangleq Q_0(\boldsymbol{s}_t, \boldsymbol{a}_t) - Q_1(\boldsymbol{s}_t) \\ \Lambda_b(\boldsymbol{s}_t, \boldsymbol{a}_t) &\triangleq D_0(t) Q_0(\boldsymbol{s}_t, \boldsymbol{a}_t) + D_1(t) Q_1(\boldsymbol{s}_t) \end{aligned} \tag{13}$$

Clearly, given the eigen long-term rewards Λ_a and Λ_b, we can recover the expected long-term rewards Q_0 and Q_1 simply by solve the equations in Eq. (13). Specifically, we have

$$\begin{aligned} Q_0(\boldsymbol{s}_t, \boldsymbol{a}_t) &= D_1(t)\Lambda_a(\boldsymbol{s}_t, \boldsymbol{a}_t) + \Lambda_b(\boldsymbol{s}_t, \boldsymbol{a}_t) \\ Q_1(\boldsymbol{s}_t) &= -D_0(t)\Lambda_a(\boldsymbol{s}_t, \boldsymbol{a}_t) + \Lambda_b(\boldsymbol{s}_t, \boldsymbol{a}_t) \end{aligned} \tag{14}$$

Therefore, once we can estimate the eigen long-term rewards Λ_a and Λ_b, we can also estimate the expected long-term rewards Q_0 and Q_1.

Although the eigen immediate/long-term rewards are just equivalent transformations of the expected immediate/long-term rewards, their iterative functions are much simplified. Actually, we have

Proposition 3.3. *[Iterative Function of Eigen Long-Term Rewards] The iterative function of the eigen long-term rewards writes*

$$\begin{aligned} \Lambda_a(\boldsymbol{s}_t, \boldsymbol{a}_t) &= \Gamma_a(\boldsymbol{s}_t, \boldsymbol{a}_t) \\ \Lambda_b(\boldsymbol{s}_t, \boldsymbol{a}_t) &= \Gamma_b(\boldsymbol{s}_t, \boldsymbol{a}_t) + \gamma \Lambda_b(\boldsymbol{s}_{t+1}, \boldsymbol{a}_{t+1}) \end{aligned} \tag{15}$$

The proof can be referred to in Appendix. A. We explain more about Proposition 3.3. Firstly, Λ_a is the difference between the long-term rewards under recommendations by real-time computation and by the cache, i.e., Q_0 and Q_1. The first equation of Eq. (15) shows that such a difference equals the difference between the immediate rewards under real-time and cached recommendations. It is because the future cache states are stochastic and independent of the current cache state. Secondly, Λ_b, which is the weighted sum of real-time and cached recommendations, follows an iterative equation independent of Λ_a.

Compared to the iterative functions of the expected long-term rewards in Eq. (11) where the two critics depend on each other, the eigen long-term rewards Λ_a and Λ_b are independent. Therefore, if we regard the eigen long-term rewards as the function to learn, we will not suffer from the critic dependency problem.

Given the abovementioned discussions, we are ready to provide the final algorithm.

3.3 The Eigenfunction Learning Algorithm

Now, we provide the CARL-EL algorithm according to the eigenfunctions defined in Section 3.2. The framework of CARL-EL is shown in Figure 5. Specifically, we regard the following functions as learnable functions:

- The immediate reward functions $V_0(\boldsymbol{s}_t, \boldsymbol{a}_t; \psi_0)$ and $V_1(\boldsymbol{s}_t; \psi_1)$, parameterized by ψ_0 and ψ_1.
- The eigen long-term reward function $\Lambda_b(\boldsymbol{s}_t, \boldsymbol{a}_t; \rho)$, parameterized by ρ.
- The policy function $\mu(\boldsymbol{s}_t; \theta)$, parameterized by θ.

In contrast, we regard the following functions as derived functions from the abovementioned learnable functions:

- The eigen immediate reward functions Γ_a and Γ_b, which can be obtained from the expected immediate reward V_0 and V_1 according to Eq. (12).
- The eigen long-term reward function Λ_a, which is equal to Γ_a according to Proposition 3.3.
- The expected long-term rewards Q_0 and Q_1, which can be derived from the eigen long-term rewards Λ_a and Λ_b according to Eq. (14).

The EL algorithm, as shown in Algorithm 1, consists of five steps, i.e. learning the immediate rewards, calculating the eigen immediate rewards, learning the eigen long-term reward, recovering the long-term rewards, and learning the policy function.

Step 1: we learn the immediate reward functions $V_0(\boldsymbol{s}_t; \boldsymbol{a}_t; \psi_0)$ and $V_1(\boldsymbol{s}_t; \psi_1)$. The loss function is

$$l(\psi) = \begin{cases} [r_t - V_0(\boldsymbol{s}_t, \boldsymbol{a}_t; \psi_0)]^2 & , C_t = 0 \\ [r_t - V_1(\boldsymbol{s}_t; \psi_1)]^2 & , C_t = 1 \end{cases} \tag{16}$$

We learn different immediate rewards for requests processed by real-time and cached recommendations, where $C_t = 0$ or 1, respectively.

Step 2: after obtaining the estimation of immediate value functions, we calculate the eigen immediate rewards $\Gamma_a(\boldsymbol{s}_t, \boldsymbol{a}_t)$ and $\Gamma_b(\boldsymbol{s}_t, \boldsymbol{a}_t)$ according to Eq. (12).

Step 3: the eigen immediate rewards Γ_a and Γ_b are used to learn the eigen long-term rewards Λ_a and Λ_b via the temporal difference function. According to Proposition 3.3, we have $\Lambda_a = \Gamma_a$, therefore,

Algorithm 1 CARL-EL: Eigenfunction Learning of Cache-Aware Reinforcement Learning

Input: $\{\boldsymbol{s}_{1:T}, \boldsymbol{a}_{1:T}, r_{1:T}\}$ for each user.
Output: A policy function $\mu(\boldsymbol{s}_t; \theta)$ parameterized by θ.
for each user session with T requests from the replay buffer **do**
  for $t = 1, \cdots, T$ **do**
    Collect the user state $\boldsymbol{s}_t$, the reward r_t, and the action $\boldsymbol{a}_t$ from the replay buffer.
    Step 1: Learn the immediate rewards $V_0(\boldsymbol{s}_t, \boldsymbol{a}_t; \psi_0)$ and $V_1(\boldsymbol{s}_t; \psi_1)$ according to Eq. (16).
    Step 2: Calculate the eigen immediate rewards $\Gamma_a(\boldsymbol{s}_t, \boldsymbol{a}_t)$ and $\Gamma_b(\boldsymbol{s}_t, \boldsymbol{a}_t)$ according to Eq. (12).
    Step 3: Calculate the eigen long-term rewards $\Lambda_a(\boldsymbol{s}_t, \boldsymbol{a}_t)$, and learn $\Lambda_b(\boldsymbol{s}_t, \boldsymbol{a}_t; \rho)$ according to Eq. (17).
    Step 4: Calculate the critic function $Q_0(\boldsymbol{s}_t, \boldsymbol{a}_t)$ according to Eq. (14).
    Step 5: Take policy gradient with the policy loss $Q_0(\boldsymbol{s}_t, \mu(\boldsymbol{s}_t; \theta))$ to update θ according to Eq. (5).
  end for
end for

there is no need to learn Λ_a. In contrast, Λ_b needs to be learned by the following temporal difference loss:

$$l(\rho) = \left[\Lambda_b\left(\boldsymbol{s}_t, \boldsymbol{a}_t; \rho\right) - \left(\Gamma_b(\boldsymbol{s}_t, \boldsymbol{a}_t) + \gamma \Lambda_b\left(\boldsymbol{s}_{t+1}, \mu(\boldsymbol{s}_{t+1}; \theta^-); \rho^-\right)\right)\right]^2 \tag{17}$$

where θ^- and ϕ^- are the parameters of the target actor and target critic, respectively.

Step 4: we recover the expected long-term reward Q_0 and Q_1 from the eigen long-term reward Λ_a and Λ_b according to Eq. (14).

Step 5: we regard the expected long-term reward $Q_0(\boldsymbol{s}_t, \boldsymbol{a}_t)$ as the policy loss, and take the policy gradient algorithm according to Eq. (5).

Figure 5 shows the information flow of Algorithm 1. Compared with the direct learning method shown in Figure 4, the main difference between Algorithm 1 and CARL-DL is the temporal difference equation, i.e. Eq. (17) in CARL-EL and Eq. (8) in CARL-DL. In CARL-DL, the learning of the critic functions Q_0 and Q_1 depend on each other according to a stochastic cache state C_t; while in CARL-EL, the learning of the eigen long-term rewards Λ_a and Λ_b does not depend on each other. Therefore, the temporal difference of CARL-EL does not rely on the stochastic cache state C_t, and hence, the variance of learning can be effectively reduced, and the performance can be improved.

4 EXPERIMENTAL RESULTS

We deploy our proposed CARL model in Kwai, a short video platform serving over 100 million users. The QPS is shown in Figure 1, and the traffic router executes as described in Section 2. The ratio of cached recommendations during peak periods is about 40%. We did not conduct offline experiments because there is, to the best of the authors' knowledge, no offline experimental environment considering the impact of the cache for the time being.

4.0.1 Implementation. The structure of the online system is shown in Figure 6, and the details are as follows:

- **Sample Generation**. We first collect the states, actions, and rewards of each request to generate request-wise samples. Next, a session collector groups the requests of the same user and orders them by the timestamp of the requests. We split the request sample list into multiple sessions based on the 15-minute inactivity rule. The session collector constructs two consecutive requests from the same session into a single RL sample and feeds them into the replay buffer.
- **MDP**. We deploy the actor in the ranking part of the recommender system. The recommender serves as the agent, and the users serve as the environment. At each user request, the actor gets the state $\boldsymbol{s}_t$ containing a vector of the user profile, the behavior history, the request context, and the statistics of candidate video features. The user profile covers the information collected in the registration, including the gender, the age, and the interests of the user. The behavior history includes the items that the user interacted with in the past. The request context includes the timestamp and the location, and the video statistics include the average score and the 10-th/30-th/50-th/70-th/90-th percentile of the prediction scores. Then, it returns an action vector $\boldsymbol{a}_t$, which is a 5-dimensional continuous vector ranging in $[0, 3]$, acting as the fusion parameters of five scoring models predicting the main feedback (watchtime, shortview, longview, finish, and forward). A final ranking score is generated according to Eq. (2), of which the top L items are selected, and the first K items are shown to the users (the red arrow in Figure 6). The rest $L - K$ items are cached in case of a cached recommendation when the traffic exceeds the system's affordability(the green arrow in Figure 6). In our scenario, $L = 40$ and $K = 8$.

4.1 Settings

4.1.1 Baselines. We deploy different methods in the online system:

- **Cross Entropy Method (CEM)** [10]: A black-box optimization method commonly used for hyper-parameter optimization. We use CEM to search the best parameters $\boldsymbol{a}_t$.
- **TD3** [3]: A famous RL method which uses twin critics to reduce the bias. TD3 does not explicitly distinguish the real-time and cached recommendations.
- **RLUR**[1]: A state-of-the-art reinforcement-learning-based method for the fusion of multiple predictions. RLUR is the last online version before CARL is deployed in Kwai. Similar to TD3, RLUR does not explicitly distinguish the real-time and cached recommendations, either.
- **CARL-DL**: The CARL with the direct learning method.
- **CARL-EL**: The CARL with the eigenfunction learning method.

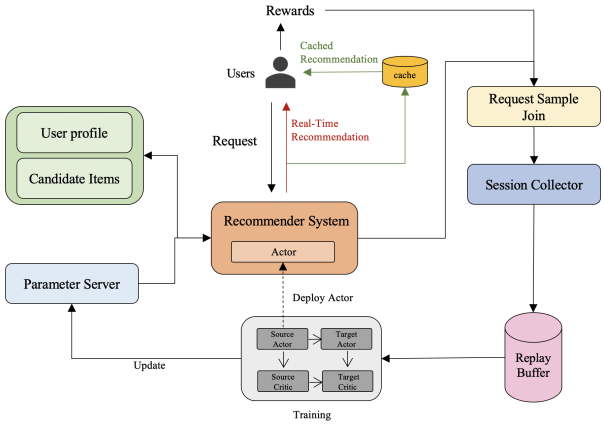


Figure 6: System Implementation

	Session Watch Time	Daily Watch Time
CEM	+0.000%	+0.000%
TD3	+0.176%	+0.184%
RLUR	+0.206%	+0.192%
CARL-DL	+0.390%	+0.342%
CARL-EL	**+0.545%**	**+0.586%**

Table 3: Experimental Results in Kwai in terms of the improvement over CEM.

4.1.2 Metrics. We evaluate the abovementioned algorithms by the session-wise watch time and the daily watch time, an important metric in short video platforms [14]. We show the experiment results in improvement compared to the CEM method.

4.2 Results

Table 3 shows the comparison results of different methods in online A/B experiments. In our scenario, even a 0.1% improvement in the watch time is significant. It is shown that RLUR and TD3 outperform CEM due to the effectiveness of RL approaches. Both the CARL-DL and the CARL-EL outperform RLUR and TD3, showing the effectiveness of the cache-aware modeling. Moreover, the CARL-EL achieves the best performance, showing the effectiveness of the proposed EL method.

4.3 Discussions

4.3.1 Improvement on Critic Loss. Table 4 shows the average critic loss of different RL-based methods over one day. We evaluate these methods' critic loss using the formula shown in Eq. (4). It is shown that CARL outperforms the other algorithms, showing the effectiveness of the explicit modeling of cache. Moreover, CARL-EL achieves a better critic loss than CARL-DL, showing that the EL approach effectively improves the performance of CARL.

4.3.2 Gaps between Recommendations by Real-Time Computation and by the Cache. Figure 7 shows the average long-term reward

Methods	TD3	RLUR	CARL-DL	CARL-EL
Loss	0.33	0.30	0.28	**0.25**

Table 4: Critic Loss of Different Methods.

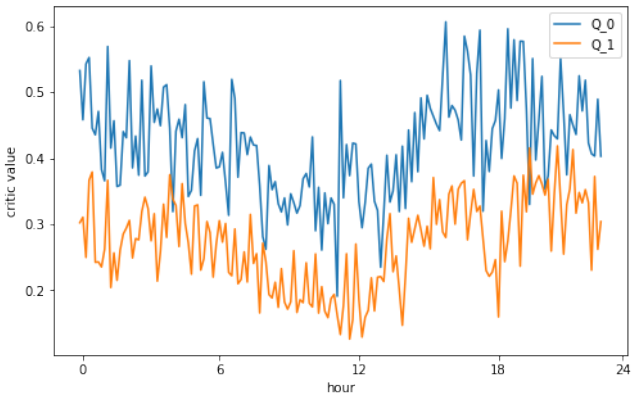


Figure 7: Comparisons of critic values.

estimation of the CARL-EL algorithm under recommendations by real-time computation and by the cache, i.e., Q_0 and Q_1, respectively. It is shown that Q_0 is consistently larger than Q_1, showing that the model successfully learns the advantage of real-time recommendations over cached recommendations. Moreover, Q_0 and Q_1 have similar trendings over one day. For example, they both reach minimal values at about 12pm, while reaching maximal values at about 18pm. It can be explained by Eq. (15), where $\Lambda_a = Q_0 - Q_1 = V_0 - V_1$, which means the difference between real-time and cached recommendations is independent of the cache ratios $D_0(t)$ and $D_1(t)$.

We further discuss the user feedback under recommendations by real-time computation and by the cache. Figure 8 shows the average watch time per video in recommendations by real-time recommendations and by the cache. Although the performance of cached recommendations is lower than that of real-time recommendations, the CARL model increases the performance of cached recommendations more than that of real-time recommendations. Such results show that effective cache modeling helps reduce the performance gap between real-time and cached recommendations.

4.3.3 Computational Burden. We test the total time cost of the recommender system under different methods. TD3 increases the time cost by 0.524% compared with CEM, while the difference in the time costs among these RL methods (TD3, RLUR, CARL-DL, and CARL-EL) is less than 0.1% because we keep actors unchanged in the experiments. Moreover, to verify the trade-off of computational burden and the recommendation performance introduced by the result cache, we test the RLUR in an experimental group without a result cache, i.e. all the requests are performed by real-time computation. Such settings increase the daily watch time by 0.642%, but with a significant increase of total time cost by 67.6%. In contrast, CARL-EL increases the daily watch time by 0.391% compared to

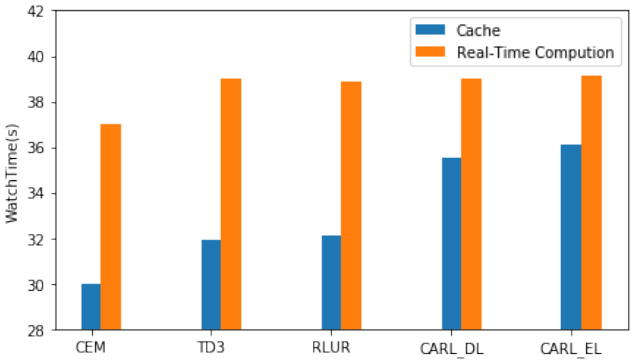


Figure 8: Average watch time per video of recommendations by real-time computation and by the cache.

RLUR with nearly no increase in the time cost, which shows the effectiveness of CARL-EL.

5 CONCLUSION

This paper proposes the CARL model to reinforce the long-term rewards in the recommender systems with a result cache. CARL uses a cache state to represent whether the recommender system performs recommendations by real-time computation or by the cache. The cache state is determined by the computational load of the recommender system. Moreover, we propose an EL algorithm to learn independent critics for CARL to solve the critic dependency. Experiments show that CARL can significantly improve the users' engagement when considering the result cache. CARL has been fully launched in Kwai app, serving over 100 million users.

REFERENCES

[1] Qingpeng Cai, Shuchang Liu, Xueliang Wang, Tianyou Zuo, Wentao Xie, Bin Yang, Dong Zheng, Peng Jiang, and Kun Gai. 2023. Reinforcing User Retention in a Billion Scale Short Video Recommender System. *arXiv preprint arXiv:2302.01724* (2023).

[2] Minmin Chen, Alex Beutel, Paul Covington, Sagar Jain, Francois Belletti, and Ed H Chi. 2019. Top-k off-policy correction for a REINFORCE recommender system. In *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining*. 456–464.

[3] Scott Fujimoto, Herke Hoof, and David Meger. 2018. Addressing function approximation error in actor-critic methods. In *International conference on machine learning*. PMLR, 1587–1596.

[4] Chongming Gao, Shijun Li, Wenqiang Lei, Jiawei Chen, Biao Li, Peng Jiang, Xiangnan He, Jiaxin Mao, and Tat-Seng Chua. 2022. KuaiRec: A fully-observed dataset and insights for evaluating recommender systems. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 540–550.

[5] Theodoros Giannakas, Pavlos Sermpezis, and Thrasyvoulos Spyropoulos. 2018. Show me the cache: Optimizing cache-friendly recommendations for sequential content access. In *2018 IEEE 19th International Symposium on" A World of Wireless, Mobile and Multimedia Networks"(WoWMoM)*. IEEE, 14–22.

[6] Biye Jiang, Pengye Zhang, Rihan Chen, Xinchen Luo, Yin Yang, Guan Wang, Guorui Zhou, Xiaoqiang Zhu, and Kun Gai. 2020. DCAF: A Dynamic Computation Allocation Framework for Online Serving System. *arXiv preprint arXiv:2006.09684* (2020).

[7] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.

[8] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2015. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971* (2015).

[9] Shichen Liu, Fei Xiao, Wenwu Ou, and Luo Si. 2017. Cascade ranking for operational e-commerce search. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1557–1565.

[10] Reuven Y Rubinstein and Dirk P Kroese. 2004. *The cross-entropy method: a unified approach to combinatorial optimization, Monte-Carlo simulation, and machine learning*. Vol. 133. Springer.

[11] Yanfeng Wang, Mingyang Ding, Zhiyong Chen, and Ling Luo. 2017. Caching placement with recommendation systems for cache-enabled mobile social networks. *IEEE Communications Letters* 21, 10 (2017), 2266–2269.

[12] Zhe Wang, Liqin Zhao, Biye Jiang, Guorui Zhou, Xiaoqiang Zhu, and Kun Gai. 2020. Cold: Towards the next generation of pre-ranking system. *arXiv preprint arXiv:2007.16122* (2020).

[13] Wanqi Xue, Qingpeng Cai, Ruohan Zhan, Dong Zheng, Peng Jiang, and Bo An. 2022. ResAct: Reinforcing Long-term Engagement in Sequential Recommendation with Residual Actor. *arXiv preprint arXiv:2206.02620* (2022).

[14] Ruohan Zhan, Changhua Pei, Qiang Su, Jianfeng Wen, Xueliang Wang, Guanyu Mu, Dong Zheng, Peng Jiang, and Kun Gai. 2022. Deconfounding Duration Bias in Watch-time Prediction for Video Recommendation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4472–4481.

[15] Gengrui Zhang, Yao Wang, Xiaoshuang Chen, Hongyi Qian, Kaiqiao Zhan, and Ben Wang. 2024. UNEX-RL: Reinforcing Long-Term Rewards in Multi-Stage Recommender Systems with UNidirectional EXecution. *arXiv preprint arXiv:2401.06470* (2024).

[16] Lixin Zou, Long Xia, Zhuoye Ding, Jiaxing Song, Weidong Liu, and Dawei Yin. 2019. Reinforcement learning to optimize long-term user engagement in recommender systems. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2810–2818.

A PROOF OF PROPOSITION 3.3

Proof. We consider the iterative function Eq. (11) of the expected long-term rewards L_0 and L_1. By taking the difference between the first equation and the second equation in Eq. (11), we have $L_0 - L_1 = V_0 - V_1$, i.e. $\Lambda_0 = \Gamma_0$. Then, by taking a weighted summation of the first and second equations in Eq. (11) with weight $D_0(t)$ and $D_1(t)$ respectively, we have

$$\begin{aligned} &D_0(t)L_0(\boldsymbol{s}_t, \boldsymbol{a}_t) + D_1(t)L_1(\boldsymbol{s}_t) \\ &= D_0(t)V_0(\boldsymbol{s}_t, \boldsymbol{a}_t) + D_1(t)V_1(\boldsymbol{s}_t) \\ &+ \gamma \left[D_0(t) + D_1(t)\right] D_0(t+1)\mathbb{E}_{\boldsymbol{a}_{t+1}} L_0\left(\boldsymbol{s}_{t+1}, \boldsymbol{a}_{t+1}\right) \\ &+ \gamma \left[D_0(t) + D_1(t)\right] D_1(t+1)\mathbb{E}_{\boldsymbol{a}_{t+1}} L_1\left(\boldsymbol{s}_{t+1}, \boldsymbol{a}_{t+1}\right) \end{aligned} \tag{18}$$

given the fact that $D_0(t) + D_1(t) = 1$, we have

$$\begin{aligned} &D_0(t)L_0(\boldsymbol{s}_t, \boldsymbol{a}_t) + D_1(t)L_1(\boldsymbol{s}_t) \\ &= D_0(t)V_0(\boldsymbol{s}_t, \boldsymbol{a}_t) + D_1(t)V_1(\boldsymbol{s}_t) \\ &+ \gamma\mathbb{E}_{\boldsymbol{a}_{t+1}}\left[D_0(t+1)L_0\left(\boldsymbol{s}_{t+1}, \boldsymbol{a}_{t+1}\right) + D_1(t+1)L_1\left(\boldsymbol{s}_{t+1}, \boldsymbol{a}_{t+1}\right)\right] \end{aligned} \tag{19}$$

which completes the proof. □